

APPUNTI OCPJP

Indice

Indice	2
Classi Java	7
I modificatori (protected, default, public e private) :	7
Instanceof	8
Cast	8
Costruttori	9
Overload	10
Override	10
Advance class design	11
Enumeration	11
Dichiarazione di enumeration con parametri.....	11
Inner class	11
Da una classe ad una innerClass.....	11
La differenza tra la classe esterna o interna (annidata).....	12
Tabella riassuntiva inner class.....	12
Object Oriented Design	13
Tabella riassuntiva uso di super e this	13
Inheritance (IS-A) vs. Composition (HAS-A) Relationship	13
Inheritance (IS-A).....	13
Composition (HAS-A).....	13
Classe astratta	13
Differenze tra classe astratta e concreta.....	13
Interfaccia (inheritance is-a)	14
Ereditarietà di interfacce (eredità multipla).....	14
Generics and collections	15
Introduzione	15
Tipi primitivi.....	15
Le Collection	15
Tipi di Collection.....	16
Collection a confronto.....	16
Metodi Equals e Hashcode	16
Il metodo equals()	16

Il metodo hashCode()	17
Comparable e Comparator	17
Collections utilities	18
Generici	18
Esempio di uso dei generici in un metodo.....	18
Esempio di uso dei generici in una classe.....	19
Quando usare il generico.....	19
Esempio uso generics:.....	20
String processing	22
Caratteri escape di java.....	22
Esempi di principali formattazioni:.....	22
Printf e formattazione di stringa<	23
illegalFormatConversionException.....	24
MissingFormatWidthException.....	24
MissingFormatArgumentException.....	24
tipi di formattazioni.....	24
Regular Expression classi Pattern eMatcher	24
La classe Pattern.....	24
La classe Matcher.....	25
Simboli regular expression riferiti al carattere.....	25
Simboli regular expression riferiti alla parola.....	25
Regular expression Quantificatori.....	25
Pattern non valido.....	25
Java supporto ai file (package NIO)	26
Classe Console	26
I metodi principali della classe Console	26
Perché Char[] password	26
Il package NIO	27
Metodo read()	27
Metodo ReadLine() solo verso buffered	27
try with resources.....	27
L'interfaccia autocloseable.....	27
Operazione su file di testo	28
La classe Scanner.....	28

File binary (stream di byte)	28
Metodo read() di n byte	28
Metodo read() di tipi primitive (es. int, char, short, long, float, double)	28
File di Oggetti (serialization)	29
ClassNotFoundException.....	29
Serialization	29
Java supporto ai file (package NIO2)	30
Classe Path implementa Comparable, Iterable e Watchable	30
Metodi dell'interfaccia Path.....	30
Comparing di due path.....	31
Classe Files	31
Walking a file tree	32
Classe che implementa interfaccia FileVisitor<? Super Path> visitor.....	32
FileVisitResult enum.....	32
PathMatcher	32
Watching a directory (WatchService)	33
Eccezioni e asserzioni	34
Tipi di eccezioni	34
A runtime (unchecked exception) controllata	34
A compiletime (checked exception) gestita	34
Modalità di gestione delle eccezione	35
Stacktrace	35
Catch multiplo e il multi-catch	36
Eccezioni concatenate	36
Come si rilancia un'eccezione al chiamante.....	36
Due modi per creare un'eccezione concatenata.....	36
IllegalStateException (se setto più di una volta la causa).....	37
Blocco Finally	37
Throw	37
Rethrow exception (Java 7)	37
Try with Resources (Java 7)	38
Eccezioni all'interno di blocchi statici e non statici	38
Flusso di esecuzione di un programma java.....	39
Custom Exception	39

Assertion	40
Building database applications with JDBC	41
Aggiungiamo un database nel nostro progetto	41
Interrogazioni del database (query)	42
Statement	42
ResultSet	42
Aggiornare il database	43
Per quanto riguarda lo statement.....	43
Per quanto riguarda il resultSet.....	44
ResultSet.updateRow(),resultSet.insertRow(),resultSet1.deleteRow();.....	44
Metadata di un database	45
Transazioni	45
ACID.....	45
SetAutoCommit().....	45
RollBack().....	45
SavePoint().....	46
Interfaccia RowSet()	46
Localization	47
Locales	47
Local più usato.....	47
Formato di un local	47
Classe ResourceBundle	47
Classe PropertyResourceBundle (traduzione di stringhe)	48
Formato file property.....	48
Classe ListResourceBundle	48
Definizione della classe ResBundle.....	49
Algoritmo per la scelta del bundle	49
Metodi della classe ResourceBundle	50
Formattazione numeri, valute e date	50
Formattazione di un numero	50
Classe Currency	51
Classe DateFormat	51
Classe SimpleDateFormat che estende la DateFormat	51
La classe Thread	53

Creazione di un Thread	53
Estendere la classe Thread.....	53
Implementare l'interfaccia Runnable.....	53
Thread e l'InterruptedException	54
I metodi della classe Thread	54
Esecuzione asincrona dei Thread	54
Ciclo di vita del Thread	54
Data races	55
Soluzione data races.....	55
Esecuzione sincrona dei thread	55
Sincronizzazione di un blocco.....	55
Sincronizzazione di un metodo.....	55
Problemi dei locks dei thread	56
DeadLocks	56
Meccanismo Wait/Notify	56
Tutti gli stati del Thread	57
IllegalThreadStateException	57
Package java.util.concurrent	58
La classe Synchronizer	58
Semaphore.....	58
CountDownLatch.....	59
Exchanger.....	59
CyclicBarrier	59
Phaser.....	60
Thread-safe	60
Variabili atomiche	61

Classi Java

I modificatori (protected, default, public e private) :

I modificatori sono keywords essenziali per l'incapsulamento della classe. Con i modificatori stabiliamo quali metodi o attributi della nostra classe devono essere visibile o non visibili:

- 1) **Private** : non visibili
- 2) **Public** : visibili a tutti
- 3) **Default** : visibili nel package
- 4) **Protected** : visibili solo alle classi derivate

Nell'esempio sotto notiamo come è importante fare attenzione ai Modificatori;

```

v [c1] package c1;
    > [Alphatest.java] public class Alphatest {
                                protected int failed = 10;
                                int passed = 50;
                                }

v [c1.sub] package c1.sub;
    > [Betatest.java] import c1.Alphatest;
    > [Main.java] public class Betatest extends Alphatest {
                                int total;
                                public void report() {
                                    total += failed;
                                }
                                public int getTotal() {
                                    report();
                                    return total;
                                }
                                }

                                package c1.sub;
                                import c1.Alphatest;
                                import c1.sub.Betatest;
                                public class Main {
                                    public static void main(String[] args) {
                                        Alphatest a = new Betatest();
                                        //System.out.println(((Betatest) a).failed);
                                        System.out.println(((Betatest) a).total);
                                        Alphatest b = new Alphatest();
                                        System.out.println("b non ha i campi passed e failed");
                                    }
                                }

```

failed → **protected** visibile solo nelle classi che ereditano da Alphatest, quindi solo la classe figlia Betatest. **Non visibile in tutte le altre classi anche se dello stesso package.**

passed → **default** visibile nel package c1 ma come puoi notare Betatest e Main sono nell'altro package c1.sub, **dove passed non sarà visibile.**

total → **default** visibile solo nel package c1.sub e quindi visibile anche nella classe Main, che si trova nello stesso package; **(non visibile in Alphatest)**

getTotal() → **public** se la classe Main si trovasse in un package diverso da c1.sub (quindi il default renderebbe total non visibile) utilizzeremo un metodo pubblico per restituire il valore di total.

Instanceof

```
x instanceof y
Figura x, y; x instanceof Figura
Quadrato extends Figura;
Rettangolo extends Figura;
```

```
x è un Quadrato; Figura x=new Quadrato();
y è un Rettangolo; Figura y=new Rettangolo();
```

```
x instanceof Figura; (un quadrato estende una figura → VERO)
y instanceof Figura; (un rettangolo estende una figura → VERO)
```

ma `x instanceof Quadrato` è **VERA** - mentre `x instanceof Rettangolo` è **FALSA** perché :

- **Staticamente e dinamicamente** `x` e `y` sono entrambi una figura;
- **a runtime** `x` resta una figura ma specializzata in un quadrato ne consegue che `x` è anche una istanza di quadrato;
- `y` è un'istanza di rettangolo dinamicamente;
- `y` resta una figura, è però specializzata in un rettangolo.

Riassumendo:

- o **Vero** : se `x` e `y` sono la stessa classe o `x` è un sottoclasse di `y`
- o **Falso** se dinamicamente `x` e `y` non corrispondono anche se sono entrambi sottoclassi di figura;
- o **Compilazione fallita** : se già staticamente `x` non deriva da `y`, allora si ha un errore in compilazione.

Cast

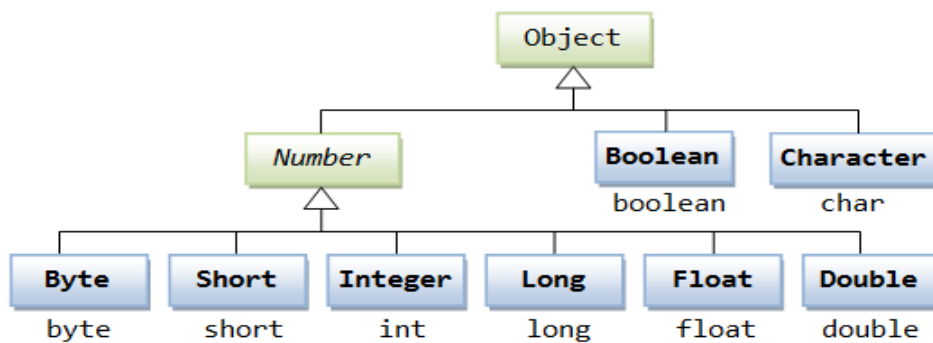
Castare un oggetto significa considerare un oggetto `x` come se fosse un oggetto `y` e questo è possibile se le due classi sono in relazione di ereditarietà.

Se la ClasseX extends ClasseY; (es. Quadrato extends Figura);
allora per il **Polimorfismo a runtime** :

```
ClasseX x=new ClasseY(); (Figura q=new Quadrato());
```

`x` allora può essere considerato (castato) un oggetto di tipo classeY perché `x instanceof ClasseY` è **VERA**.

Nell'esempio precedente se `q instanceof Quadrato` è **VERA** → ne consegue che è possibile castare `q` ad un quadrato ((Quadrato) q).



Costruttori

Il costruttore è un metodo speciale che viene richiamato ogni volta che istanziamo una classe e creiamo il nostro oggetto; ecco perché il costruttore è il posto ideale nel quale valorizzare gli attributi di una classe.

Qualora non fossero valorizzati comunque i nostri attributi assumeranno un valore di default, perciò è bene inizializzare sempre tutti gli attributi ai valori di partenza stabiliti dal programmatore.

La prima istruzione del costruttore può essere:

- `this` per riferirsi all'istanza corrente (se non eredita)
- `super` per riferirsi all'istanza della classe padre; (se eredita)

Una classe figlia eredita sempre dalla classe padre, questo si traduce in una chiamata (`super()`) nel costruttore della classe figlia, mantenendo saldo il principio di ereditarietà con la classe padre.

Questa chiamata può essere implicita o esplicita.

Keyword super Implicita

Il costruttore della classe figlia non presenta l'istruzione `super`; ma è come se ci fosse l'istruzione `super()`; che è una chiamata al costruttore di default della classe padre che in questo caso siamo obbligati a definire.

Keyword super Esplicita

Il costruttore della classe figlia presenta l'istruzione `super` con o senza parametri `super()`; o `super( args )`.

This e Super insieme

```

class Word {
    Word(String s) {
    }
}
class Buzzword extends Word {
    public Buzzword() {
        this("no parameter");
    }
    public Buzzword(String s) {
        super(s);
    }
}

```

In questo caso nel costruttore Buzzword() non richiama implicitamente super(); perché lascia l'incarico al costruttore Buzzword(String s) di farlo. Se non ci fosse super(s) verrebbe richiamato quello di default; e in tal caso ci viene segnalato un errore di compilazione che ci obbliga a definire esplicitamente un costruttore senza parameri per la classe Word.

Overload

Un metodo è overload quando si trova nella stessa classe ed ha lo stesso nome ma una differente firma.

La firma di un metodo è il nome del metodo con i suoi argomenti, é escluso il ritorno e le eccezioni.

Scelta di un metodo in overload

- Stessa firma e numero di parametri
- Se i parametri sono primitive (char, byte, short, int, long, float, double) si vede l'upcast delle primitive
- Se non esiste upcast di primitive si prende la WrapperClass o la sua superclasse, es. se ho un metodo con il parametro int e non esiste un metodo con il parametro int o il suo upcast; prenderò il metodo con il parametro Integer o con la superclasse di Integer; ovvero un Number
- Se i parametri sono delle referenziate si prende il metodo con la stessa referenziata o con l'UpCast dei parametri (superclassi)
- Se non esiste nessun Upcast si prende object

gli argomenti dei metodi :

- se sono delle primitive o stringhe vengono passati per valore; non modificano il valori nel chiamante
- se sono referenziate o array vengono passati per riferimento, modificano il valore nel chiamante.

Override

Un metodo è override quando nella classe figlia hai un metodo con lo stesso nome e la stessa firma della classe padre, ovvero con lo stesso numero e tipo di parametri.

Il metodo della classe figlia deve però avere

- la stessa visibilità o la visibilità più ampia del metodo della classe padre
- Lo stesso tipo di ritorno o una sottoclasse del tipo di ritorno della classe padre

- La stessa eccezione, nessuna eccezione o un'eccezione che è sottoclasse dell'eccezione lanciata dalla classe padre

N.B. Nel polimorfismo a runtime `classX x= new classY()` con `classY` che estende `classX`:

- Se i metodi in override sono dinamici ha priorità il metodo dinamico.

- Se i metodi in override sono statici ha priorità il metodo statico.

Per quanto riguarda gli attributi, ha sempre priorità l'attributo statico.

Advance class design

Enumeration

Lo scopo principale delle enumerazione è definire un tipo di dato che può assumere solo dei tipi prefissati di valori; puoi considerare una enumerazione una costante che ha un margine di variabilità.

Come si dichiara l'enumeration:

```
public enum Direction { NORTH, EAST, SOUTH, WEST }
```

come si legge una enumeration :

```
for (Direction d : Direction.values()) {
    system.out.print(d+"-");
}
```

Questo stamperà NORTH-EAST-SOUTH-WEST-

Dichiarazione di enumeration con parametri

```
enum MilesFromBoston {
    NEW_YORK(224.60), SAN_FRANCISCO(3097.7), DENVER(1969.0);
    Public final double distance;

    private MilesFromBoston(double distance) {
        // E' simile ad un costruttore di classe ma le istanze sono generate // solo dentro l'enumeration stessa
        this.distance = distance;
    }
}
```

Inner class

Sono classi scritte all'interno di altre classi che la utilizzano.

Per richiamare una inner class bisogna specificare:

- la classe esterna alla quale appartengono prima del punto, in caso di inner class statiche

- istanziare la classe esterna prima del punto e successivamente istanziare la classe interna, in caso di inner class dinamiche

Da una classe ad una innerClass

<pre> public class Tubes { public static void main(String[] args) { doStuff(); } static void doStuff() { new InnerTube().doInner(); } } class InnerTube { void doInner() { System.out.print("2 "); } } </pre>	<pre> public class Tubes { class InnerTube { void doInner() { System.out.print("2 "); } } } public static void main(String[] args) { doStuff(); } static void doStuff() { new Tubes().new InnerTube().doInner(); } } </pre>
--	--

La differenza tra la classe esterna o interna (annidata)

Il modificatore per una classe interna a differenza di una comune classe può anche essere static; facendo però attenzione ad alcuni accorgimenti.

Non potendo istanziare una classe static, non potrò accedere ai campi e metodi dinamici dichiarati al suo interno.

Anche per una classe dinamica interna ci sono delle differenze da evidenziare:

I campi e i metodi statici devono essere final perché non ha senso dichiarare un campo statico che è soggetto alla perdita del valore da un'istanza all'altra; se lo dichiariamo final il problema è risolto; ovvio che questo campo statico final si comporterà come una costante.

Tabella riassuntiva inner class

	CAMPO STATICO	METODO STATICO	CAMPO DINAMICO	METODO DINAMICO
CLASSE STATICA	SI	SI	ISTANZIABILE DENTRO LA INNER CLASS	ISTANZIABILE DENTRO LA INNER CLASS
			SOLO CHE ESSENDO DINAMICI POSSO VEDERLI IN UNA CLASSE DINAMICA: IStanZIABILE....	
CLASSE DINAMICA	SOLO FINAL	NO	SI	SI

Object Oriented Design

Tabella riassuntiva uso di super e this

	THIS	SUPER	ISTANZA
METODO STATICO	NO	NO	NECESSARIA
METODO DINAMICO	SI	SI	THIS PER RIFERIRSI ALL'ISTANZA DELLA STESSA CLASSE SUPER PER RIFERIRSI ALLA CLASSE CHE ESTENDE (SUPERCLASSE)

Inheritance (IS-A) vs. Composition (HAS-A) Relationship

Inheritance (IS-A)

Se un oggetto di classe A eredita da una super classe B allora in realtà la **classe A è una classe specializzata di B.**

Esempio: Un Cane è anche un animale; **SI VERO** allora possiamo partire dalla definizione di un animale ed aggiungere le caratteristiche che ci fanno distinguere l'animale cane da altri tipi di animali

Composition (HAS-A)

Se un oggetto è formato dall'unione di più oggetti, ovvero dalla composizione di più oggetti; **una classe A ha una classe B.**

Esempio: l'auto ha 4 ruote; l'auto in realtà non è una ruota ma una ruota è un oggetto che fa parte dell'auto che unito ad altri oggetti mi permette di creare un oggetto auto.

Classe astratta

La classe astratta è come un template per le classi concrete; non si istanzia proprio perché non serve a definire un oggetto concreto ma è la base per la definizione di altri oggetti concreti simili, come ad esempio la classe Figura astratta che in realtà è un template di come devono essere definite le classi rettangolo, quadrato, ecc... che sono tutte sotto classi concrete di figura.

Differenze tra classe astratta e concreta

Classe astratta	Classe concreta
Metodi con o senza corpo	Solo metodi con un corpo
Non si istanzia	istanziabile
Non devono obbligatoriamente implementare né metodi di altre classi che estende e né di interfacce che implementa. Tuttavia può farlo così la prima classe concreta che eredita non sarà obbligata ad implementare	La prima classe concreta che estende o implementa un'interfaccia deve implementare i metodi della classe che estende o i metodi delle interfacce che implementa se non sono stati ancora implementati.

Interfaccia (inheritance is-a)

L'interfaccia è un insieme di comportamenti che il nostro oggetto assume quando è sottoposto a dei comandi ai quali ovviamente, esso, è in grado di rispondere.

Le interfacce sono come delle classi astratte con alcune differenze.

Se l'oggetto di classe A è in grado di fare determinate cose descritte nell'interfaccia B allora la classe A di oggetti sono di tipo interfaccia B.

Esempio: Se definiamo come frenante la capacità di ridurre la velocità di un oggetto; l'auto che implementa l'interfaccia frenante acquisisce la capacità di fermarsi perciò diremo che l'auto è un oggetto frenante.

Ereditarietà di interfacce (eredità multipla)

Ricorda che un oggetto non può ereditare da più oggetti ma può implementare più interfacce e poi ricorda che un'interfaccia può ereditare da più interfacce.

Class A	interface A	interface A
Class B	interface B	interface B
Class C extend A,B {	interface C extend A,B;	class C implements A,B
}		

i campi statici o dinamici di una interfaccia devono essere costanti (e quindi di tipo final); infatti di solito hanno chiamate di tipo statico; senza istanza.

Classe.campo:

- i metodi dinamici sono senza corpo
- i metodi di default (default introdotto nel java 8) sono gli unici ad avere un corpo. I metodi di default hanno un corpo che occorre quando la classe che implementa questa interfaccia non ha un corpo per questo metodo.
- I metodi statici sono metodi usati solo all'interno dell'interfaccia

```
Default-interface Frenabile {
```

```
    // Default è omesso (non c'è ma è come se ci fosse); quindi un'interfaccia senza modificatore è visibile solo dentro al proprio package.
```

```
}
```

Generics and collections

Introduzione

Il linguaggio JAVA è un linguaggio fortemente tipizzato; ovvero quando dobbiamo immagazzinare dei dati in memoria (variabili) è necessario sapere che tipo di dato stiamo memorizzando; il connubio generics/collections consente al compilatore Java di accorgersi a compile time se ci sono anomalie di tipo.

Non siamo obbligati a utilizzare i generics, tuttavia per evitare possibili o quasi certi errori di tipo a runtime, è buona norma sapere che una collections conterrà dati omogenei di un certo tipo; quindi è buona norma usare i generics.

```
List<(reference type)> miaLista = new ArrayList<>();
```

Reference type quindi non sono ammessi i tipi primitivi che sono sostituiti dalle rispettive Wrapperclass.

Tipi primitivi

Primitive Type	Size	Minimum Value	Maximum Value	Wrapper Type
char	16-bit	Unicode 0	Unicode 2 ¹⁶ -1	Character
byte	8-bit	-128	+127	Byte
short	16-bit	-2 ¹⁵ (-32,768)	+2 ¹⁵ -1 (32,767)	Short
int	32-bit	-2 ³¹ (-2,147,483,648)	+2 ³¹ -1 (2,147,483,647)	Integer
long	64-bit	-2 ⁶³ (-9,223,372,036,854,775,808)	+2 ⁶³ -1 (9,223,372,036,854,775,807)	Long
float	32-bit	32-bit IEEE 754 floating-point numbers		Float
double	64-bit	64-bit IEEE 754 floating-point numbers		Double
boolean	1-bit	true or false		Boolean

La stringa non è un dato primitivo e può essere utilizzata come generics.

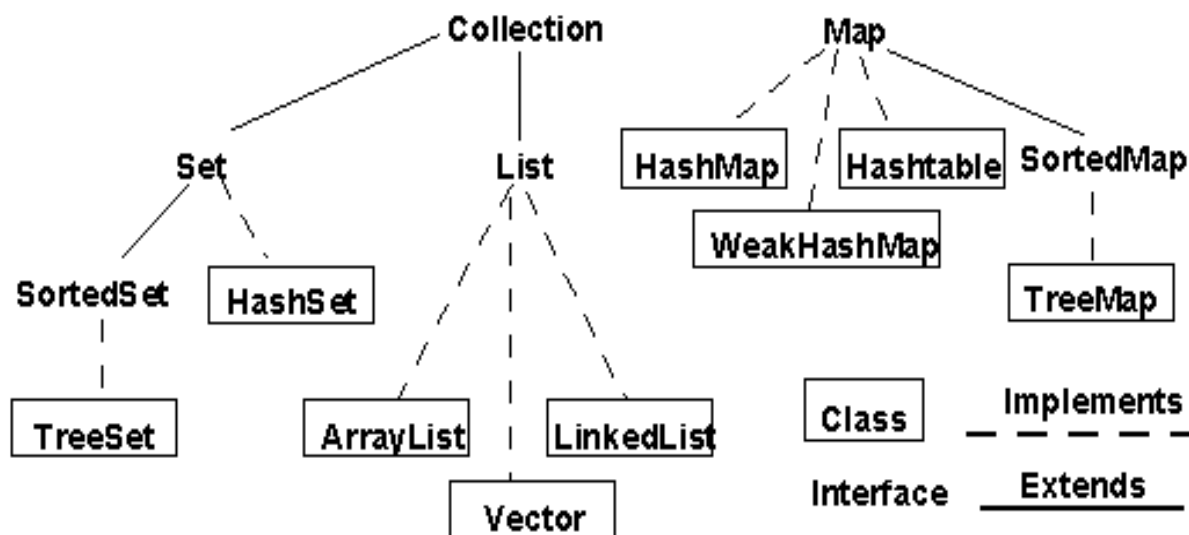
Le Collection

Il tipo di iterazione che possiamo avere con una lista ci consente di definire una serie di tipi di liste:

- L'ordine in cui vengono inseriti i dati
- L'approccio alla ricerca di un elemento nella lista
- La possibilità di avere elementi duplicati

Questi fattori danno origine a vari tipi di liste ognuna delle quali è più efficiente per il nostro programma, a seconda appunto dell'iterazione che si vuole avere con la lista.

Tipi di Collection



Collection a confronto

	ArrayList	Vector	LinkedList	HashMap	LinkedHashMap	HashTable	TreeMap	HashSet	LinkedHashSet	TreeSet
Allows Null?	Yes	Yes	Yes	Yes (But One Key & Multiple Values)	Yes (But One Key & Multiple Values)	No	Yes (But Zero Key & Multiple Values)	Yes	Yes	No
Allows Duplicates?	Yes	Yes	Yes	No	No	No	No	No	No	No
Retrieves Sorted Results?	No	No	No	No	No	No	Yes	No	No	Yes
Retrieves Same as Insertion Order?	Yes	Yes	Yes	No	Yes	No	No	No	Yes	No
Synchronized?	No	Yes	No	No	No	Yes	No	No	No	No

Metodi Equals e Hashcode

L'Override di questi due metodi ci consente di definire come due oggetti devono essere confrontati; se questi due metodi non sono definiti è impossibile stabilire se due oggetti sono uguali.

Questi metodi devono assolutamente essere implementati quando facciamo uso di collection che non ammettono duplicati.

Non implementando questi metodi non sarà lanciata alcuna eccezione ed il programma compilerà senza problemi ma Java non riuscirà a stabilire se due elementi nella collection sono uguali; accettando così i duplicati.

Il metodo equals()

Con questo metodo definite le condizioni per le quali due oggetti devono considerarsi uguali. (es. per un cerchio che abbiano lo stesso raggio)

Questo metodo applica una relazione di equivalenza che:

- E' riflessiva: `x.equals(x)` è sempre true.
- E' simmetrica: `x.equals(y)` e `y.equals(x)` restituiscono entrambi true.
- E' transitiva: se `x.equals(y)` e `y.equals(z)` sono true, allora anche `x.equals(z)` deve restituire true.
- E' consistente: se gli attributi utilizzati per calcolare l'uguaglianza sono gli stessi, chiamate multiple a `x.equals(y)` devono restituire sempre true.
- `x.equals(null)` deve restituire false

Nel caso della classe `Object`, `x.equals(y)` equivale a `(x==y)` solo se puntano allo stesso oggetto.

Il metodo hashCode()

Restituisce un valore intero rappresentante l'hash per un oggetto. E' usato in tanti ambiti, per esempio nelle hashmap.

Le sue peculiarità sono:

- Consistente: Una volta istanziato l'oggetto, il valore restituito da questo metodo non deve cambiare sempre se non cambino i campi con cui viene calcolato.
- Valori diversi: Istanze dello stesso oggetto su diverse esecuzioni dell'applicazioni devono invece restituire valori diversi.
- Nel metodo equals: Se due oggetti sono dichiarati uguali allora la funzione hash deve restituire lo stesso valore.

Non è obbligatorio che due oggetti che risultano diversi nel metodo `equals()` devono avere un hashcode diverso, ma è comunque consigliato, per questioni di performance.

N.B. HashSet e HashMap accetta elementi nulli ed aggiungono elementi nella collection in modo casuale.

Comparable e Comparator

Non confondere i metodi `equals` e `hashCode` con `comparable`. Innanzitutto parliamo d'interfaccia `comparable` e di solito viene usata nelle collections `TreeSet` e `TreeMap` per ordinare gli elementi.

La classe che ha una collection `TreeSet`, `TreeMap` deve implementare per forza l'interfaccia `comparable` altrimenti si ha un'eccezione a runtime:

Exception in thread "main" java.lang.ClassCastException:
<<nome della classe>> cannot be cast to java.lang.Comparable

N.B. Le collection che implementano comparable non accettano elementi nulli, proprio perché non sarà possibile ordinare un elemento nullo...

Nella collection treeMap il Comparable agisce sulla chiave e perciò gli elementi saranno ordinati per chiave, implementando l'interfaccia comparator si ha anche la possibilità di ordinare la collection treeMap su altri attributi.

```
public class Dipendente implements Comparable<Dipendente> {
    String matricola, nome, cognome;
    Long dataAssunzione;

    // in questo caso la treeMap è ordinata per chiave e nel nostro caso la chiave è la matricola

    public class ComparatorCognomeNome implements Comparator<Dipendente>{
        @Override
        public int compare(Dipendente arg0, Dipendente arg1) {
            if (arg0.cognome!=arg1.cognome)
                return arg0.cognome.compareTo(arg1.cognome);
            return arg0.nome.compareTo(arg1.nome);
        }
    }

    // in questo caso la treeMap * è ordinata per cognome e se il cognome è uguale verrà confrontato il nome.
```

Collections utilities

```
Collections.sort(list, null); <- ordina gli elementi in base alle chiavi.
Collections.sort(list, ComparatorCognomeNome); <- ordina gli elementi
usando un comparator
Collections.binarySearch <- richiede che gli elementi siano ordinati
```

Generici

Il linguaggio java ha molti vantaggi nel suo stile di programmazione ma i suoi punti di forza sono:

- la capacità di rilevare gli errori durante la compilazione
- la riusabilità del codice.

I generici giocano un ruolo fondamentale in queste due caratteristiche del linguaggio. I generici sono stati introdotti nella versione 1.5 di java e il loro scopo è determinare il tipo di dato contenuto in un elemento della collection a compile time; quindi il grande vantaggio dei generici è stabilire prima che un programma venga eseguito se nel codice ci siano violazioni di tipo.

```
List lista=new ArrayList(); // non fa uso dei generic
Lista.add(new String("Ciao"));
Lista.add(new Double(3.0));
/*
    nessun errore neanche a runtime ma se io voglio sommare tutti gli elementi della lista?
*/
List<Integer> lista=new ArrayList<>();
Lista.add(new Double(3.0)); // errore di compilazione
/*
    Nel mio codice considero gli elementi della lista solo Integer e applicherò tutti i metodi e le operazioni
    consentite sugli interi
*/
```

L'uso del generico, inoltre, ci consente di riusare una classe o un metodo specificando un segnaposto <T>. T nel bytecode é un Oggetto generico che a runtime sarà sostituito dal tipo corretto.

Il generico non sempre è possibile, ovvero non possiamo garantire che un metodo o una classe valga per qualsiasi reference.

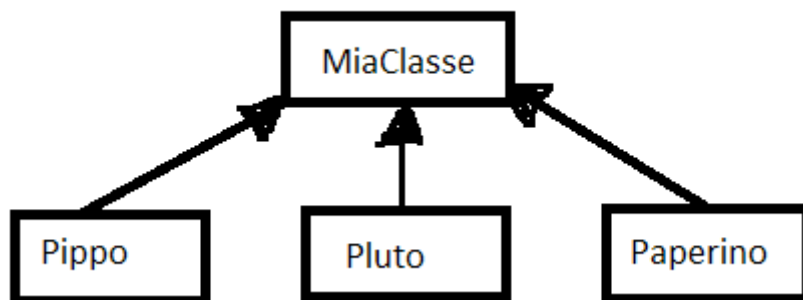
Le wildCards sono state introdotte, allora, per permettere ad un metodo o classe di agire in un range di reference:

Esempio di uso dei generici in un metodo

<? extends MiaClasse> possiamo dire che il nostro metodo agisce sulla reference MiaClasse e tutti le sottoclassi di MiaClasse (agisce anche sulla classe Pippo se Pippo extends MiaClasse).

```
Class MiaClasse {  
}  
Class Pippo extends MiaClasse {  
}  
metodo(Collection<? extends MiaClasse>) {  
}
```

Per quanto riguarda le liste non è possibile aggiungere elementi di tipo Pippo in una lista che contiene in realtà elementi che sono sotto classe di MiaClasse...



Pur essendo MiaClasse Upper bound; ovvero che accetta tutte le sottoclassi di MiaClasse; non è possibile stabilire con certezza se gli elementi aggiunti saranno o di tipo Pippo o di tipo Pluto o di tipo Paperino violando la consistenza del generics stesse...

<? Super MiaClasse> stabiliamo un confine lower bound, possiamo dire che il nostro metodo agisce sulla reference MiaClasse e tutte le classi padri di MiaClasse (agisce sulla classe Pippo se MiaClasse extends Pippo)

```
Class MiaClasse extends Pippo {  
}  
Class Pippo {  
}  
metodo(Collection<? super MiaClasse>) {  
}
```

Esempio di uso dei generici in una classe

<E extends MiaClasse> l'elemento E della classe X può essere qualsiasi dato di tipo MiaClasse o una sua sottoclasse

```
Class X <E extends MiaClasse> {  
    MiaClasse e;  
}  
  
Class MiaClasse {  
}  
Class Pippo extends MiaClasse {  
}
```

<E super MiaClasse> L'elemento E della classe X può essere qualsiasi dato di tipo MiaClasse o una sua superclasse

```
Class X <E super MiaClasse> {  
    MiaClasse e;  
}  
  
Class MiaClasse extends Pippo {  
}  
Class Pippo {  
}
```

Quando usare il generico

La programmazione con i generici è detta polimorfismo parametrico, dove il tipo effettivo di un attributo di classe è passato al codice come parametro. Non va confuso con l'altro tipo di polimorfismo riscontrato spesso in programmazione, che è il polimorfismo per inclusione, e che si ottiene usando i meccanismi di ereditarietà tra tipi.

Il polimorfismo parametrico

Quando del codice ammette diversi tipi, allora il generico ci consente di sostituire il tipo variabile con un tipo noto dichiarato successivamente nella chiamata di un metodo o nella istanziazione di una classe.

Inoltre possiamo decidere di far corrispondere al tipo variabile solo un range di tipi noti per i quali il codice contenuto nel metodo o nella classe è in grado di funzionare. (mediante l'uso di wildcard).

il polimorfismo per inclusione

Quando del codice ammette solo un tipo compatibile; ciò vuol dire che se in quella parte di codice proviamo ad inserire un tipo differente andremo senz'altro incontro ad eccezioni a runtime (tipo unchecked); dovuti ad una cattiva programmazione.

Esempio di uso dei generics:

Se vogliamo rappresentare con una classe una carta da gioco qualsiasi, sia questa una carta napoletana o una carta francese, abbiamo la necessità di avere un attributo seme e valore della carta; ma mentre per le carte napoletane il tipo di seme sono: denari, spade, coppe e bastoni; per le carte francesi il tipo di seme sono: cuori, fiori, quadri e picche.

Quindi abbiamo la necessità di avere due classi per ciascun tipo di mazzo di carte:

- il mazzo carte napoletane

- il mazzo carte francesi

In realtà, però, entrambi sono un mazzo di carte generico dove basta cambiare il tipo di seme e il valore delle carte.

Il generico **staticamente** è rappresentato da un oggetto, che fa da segnaposto al tipo variabile ma solo **a runtime** il generico verrà sostituito con il reale tipo.

In questo modo, ora, tutte le operazioni valide per un qualsiasi tipo di mazzo di carte, anche se sono mazzi di carte diversi potranno essere descritti dalla stessa classe, sarà sufficiente passare i parametri di seme e valore (MazzoDiCarte<Seme, Valore>) e a runtime avremo il corrente tipo di mazzo di carte.

```
package Enum;
public interface Semi {
}
```

```
package Enum;
public interface Valori {
}
```

```
public enum SemeNapoletane {
    Denari, Spade, Bastoni, Coppe;
}
public enum ValoreNapoletane {
    Uno, Due, Tre, Quattro, Cinque, Sei, Sette, Donna, Cavallo, Re;
}
```



```
public enum SemeFrancese {
    Cuori, Fiori, Picche, Quadri;
}
public enum ValoreFrancese {
    UNO, DUE, TRE, QUATTRO, CINQUE, SEI, SETTE, OTTO, NOVE, DIECI, JACK, DONNA, RE, JOLLY;
}
```

Francesi



```
package Main;

public class Carte<Semi, Valori> {
    Semi seme;
    Valori valore;

    Carte(Semi seme, Valori valore) {
        this.seme=seme;
        this.valore=valore;
    }

    @Override
    public String toString() {
        if (valore.toString()=="JOLLY")
            return valore.toString();
        return valore+" di "+seme;
    }
}
```

```
}
```

```
package Main;
import java.util.ArrayList;

public class MazzoDiCarte<T> extends ArrayList<T>{
}
```

```
package Main;

import Enum.Semi;
import Enum.SemiFrancesi;
import Enum.SemiNapoletane;
import Enum.Valori;
import Enum.ValoriFrancesi;
import Enum.ValoriNapoletane;

public class FactoryCarte {

    public static MazzoDiCarte<Carte<? extends Semi, ? extends Valori>> getMazzo(String tipo) {

        switch (tipo.toLowerCase()) {
            case "napoletane": {
                MazzoDiCarte<Carte<? extends Semi, ? extends Valori>> carte = new MazzoDiCarte<>();
                for (SemiNapoletane s : SemiNapoletane.values()) {
                    for (ValoriNapoletane v : ValoriNapoletane.values()) {
                        carte.add(new Carte<SemiNapoletane, ValoriNapoletane>(s, v));
                    }
                }
                return carte;
            }

            case "francesi": {
                MazzoDiCarte<Carte<? extends Semi, ? extends Valori>> carte = new MazzoDiCarte<>();
                for (SemiFrancesi s : SemiFrancesi.values()) {
                    for (ValoriFrancesi v : ValoriFrancesi.values()) {
                        carte.add(new Carte<SemiFrancesi, ValoriFrancesi>(s, v));
                    }
                }
                return carte;
            }
        }
        return null;
    }

}
```

```
package Main;

import Enum.Semi;
import Enum.Valori;

public class Programma {

    public static void main(String[] args) {
```

```

MazzoDiCarte<Carte<? extends Semi, ? extends Valori>> mazzo=
    FactoryCarte.getMazzo("napoletane");

for (Carte<? extends Semi, ? extends Valori> c: mazzo) {
    System.out.println(c);
}

mazzo=FactoryCarte.getMazzo("francesi");

for (Carte<? extends Semi, ? extends Valori> c: mazzo) {
    System.out.println(c);
}
}
}

```

Una volta creato le interfacce semi e valori, implementate da uno specifico seme o valore posso ricondurre una carta ad un generico semi e un generico Valori. Ora sarà sufficiente riempire il mazzo di carte con le carte napoletane o quelle francesi.

String processing

E' impossibile pensare ad una stringa senza una sua formattazione.

E' altrettanto impossibile pensare ad un programma senza stringhe, perciò lo studio della formattazione di una stringa è fondamentale per un buon programmatore.

La formattazione di una stringa dipende dal tipo di dato che contiene, essenzialmente esistono tre tipi di dati universalmente conosciuti:

- il numerico
- l'alfanumerico
- e la data.

Principalmente con la formattazione di una stringa :

- Diamo alla nostra stringa uno spazio ben preciso all'interno dello schermo, grazie anche al supporto dei codici escape, principalmente diciamo quanti caratteri deve occupare la nostra stringa.
- Definiamo un modello di come vogliamo che la nostra stringa deve apparire sullo schermo, questo modello detto pattern viene utilizzato per verificare il corretto inserimento della stringa mediante l'utilizzo di regular expression.

Caratteri escape di java

| Sequenza | Azione |
|------------------|--|
| <code>\n</code> | <i>Va a capo</i> |
| <code>\t</code> | <i>Tabulazione</i> |
| <code>\b</code> | <i>Cancella a sinistra</i> |
| <code>\r</code> | <i>Return, carriage return</i> |
| <code>\f</code> | <i>form feed</i> |
| <code>\\</code> | <i>Stampa il carattere \</i> |
| <code>\'</code> | <i>Stampa il carattere '</i> |
| <code>\"</code> | <i>Stampa il carattere "</i> |
| <code>\d</code> | <i>Il carattere è espresso in codice ottale</i> |
| <code>\xd</code> | <i>Il carattere è espresso in codice esadecimale</i> |
| <code>\ud</code> | <i>Il carattere è rappresentato in unicode</i> |

Esempi di principali formattazioni:

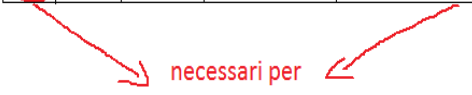
| | |
|----------------------|---|
| <code>"%15s"</code> | Riserva alla stringa 15 caratteri, posiziona la stringa a destra dello spazio riservato |
| <code>"%-15s"</code> | Riserva alla stringa 15 caratteri, posiziona la stringa a sinistra dello spazio riservato |
| <code>"%d"</code> | La stringa è formata da numeri, e riserva al numero lo spazio necessario. |
| <code>"%4d"</code> | Il numero potrà avere massimo 4 cifre, se ha meno cifre posiziona il numero a destra, lasciando spazi vuoti a sinistra |
| <code>"%04d"</code> | Come il caso precedente però anziché lasciare spazi vuoti, lascia tanti 0 quanti sono gli spazi vuoti. Dopo lo zero devo sempre specificare quanti spazi considerare. |

| | |
|-----------|--|
| "%2.f" | Formatta i numeri con la virgola fino ad un massimo di 2 decimali |
| "%10.2f" | Formatta i numeri con la virgola fino ad un massimo di 2 decimali; l'intero valore compresi i decimali non deve superare i 10 caratteri, se ci sono più cifre sarà tagliata la parte decimale. |
| "%010.2f" | Come il caso precedente ma gli spazi vuoti vengono sostituiti con lo zero (padding di 0) |

N.B. IL padding di 0 è l'unico consentito, non è possibile fare il padding con un altro numero: inoltre non è possibile fare un padding di un oggetto allineato a sinistra (%0-10s)

Printf e formattazione di stringa<

```
printStream printf(String format, Object...args)
printf("%d %c %b", 5, '2', true);
```

| | | | | | |
|---|-------|-------|------------|--------------------|---|
| % | flags | width | .precision | datatype_specifier | <p>Per formattare un oggetto sono necessari il simbolo% e il tipo da formattare; dopodiché possiamo decidere:</p> <ul style="list-style-type: none">-ampiezza riservata per quell'oggetto-Precisione per un numero float-(flags) determina l'allineamento (-) o il riempimento degli spazi (0). |
|  | | | | | |
| | | | | | |

System.out.printf("There are %3\$s %2\$s and %d truck", 5, "auto", 2+7);

Se metto \$ scelgo la posizione del valore da considerare

Se non metto nulla %d segue l'ordine naturale da 1° in poi.

E' molto importante quando formatto una stringa, rispettare il tipo di oggetto perché un numero intero o con virgola come nel caso di %d o %f, possono accettare rispettivamente o solo numeri interi o con virgola.

illegalFormatConversionException

Se i tipi non sono rispettati sarà lanciata un eccezione a runtime di tipo **IllegalFormatConversionException**:

1. non posso mettere ad un formato numerico %d → "5" una stringa.
2. non posso mettere un valore intero per una formattazione decimale %f → 5 e viceversa.

posso però tranquillamente mettere al posto di una stringa %3\$s → un valore numerico 2+7 = 9, in tal caso considererebbe il numero come una stringa.

MissingFormatWidthException

%0.2f ho deciso di fare un padding di zero ma non ho stabilito di quanti caratteri deve essere il padding perché non ho specificato l'ampiezza.

%[flags][width][.precision]datatype specifier

MissingFormatArgumentException

Quando non ci sono argomenti sufficienti nella printf

- 1) se ci sono più formattazioni (%type) di argomenti allora da l'eccezione MissingFormatArgumentException.
- 2) se ci sono meno formattazioni(%type) e più argomenti quelli in più non li stampa.

tipi di formattazioni

| Conversion Character | Type | Example |
|----------------------|---|---------------|
| d | Decimal integer | 159 |
| x | Hexadecimal integer | 9f |
| o | Octal integer | 237 |
| f | Fixed-point floating-point | 15.9 |
| e | Exponential floating-point | 1.59e+01 |
| g | General floating-point (the shorter of e and f) | — |
| a | Hexadecimal floating-point | 0x1.fccdp3 |
| s | String | Hello |
| c | Character | H |
| b | boolean | true |
| h | Hash code | 42628b2 |
| tx | Date and time | See Table 3–7 |
| % | The percent symbol | % |
| n | The platform-dependent line separator | — |

Regular Expression classi Pattern Matcher

La classe Pattern

L'insieme di espressioni regolari ci permettono di definire un modello, pattern che da una formattazione alla nostra stringa, ci dice come la stringa deve essere visualizzata e cosa deve rispettare per incontrare (matcher) la nostra espressione regolare.

La classe Matcher

Una volta deciso il nostro pattern tutte le parti della stringa che rispettano questo pattern (find) saranno le porzioni di stringa prese in considerazione (group).

I metodi della classe Matcher infatti sono:

- Matcher.find() Ritorna vero se c'è un match
- Matcher.group() Ritorna la porzione di stringa che ha rispettato il pattern
- Matcher.replaceAll() Sostituisce le porzioni di stringa (matcher.group) con una stringa.

Matcher.matches() Se il pattern incontra esattamente la regular expression e non solo una porzione di stringa

Esempio

String str = "Danny Doo, Flat no 502, Big Apartment, Wide Road, Near Huge Milestone, Hugo-city 56010, Ph: 9876543210, Email: danny@mvworld.it. Maggi Myer, Post bag no 52, Big bank post office, Big bank city 56000, ph: 9876501234, Email: maggi07@myuniverse.com";

```
Pattern pattern = Pattern.compile("\\w+@\\w+\\.\\w{1,3}"); // email
Matcher matcher = pattern.matcher(str);
while (matcher.find()) { // analogia hasNext(); se esiste un'altra // parola
    System.out.println(matcher.group()); // stampa e vai alla successiva
                                // parola, analogia con
                                // Next()
}
```

Simboli regular expression riferiti al carattere

| Symbol | Description |
|-----------------------|---|
| <code>^expr</code> | Matches expr at beginning of line. |
| <code>expr\$</code> | Matches expr at end of line. |
| <code>.</code> | Matches any single character (except the newline character). |
| <code>[xyz]</code> | Matches either x, y, or z. |
| <code>[p-z]</code> | Specifies a range. Matches any character from p to z. |
| <code>[p-z1-9]</code> | Matches either any character from p to z or any digit from 1 to 9 (remember, it won't match p1). |
| <code>[^p-z]</code> | '^' as first character inside a bracket negates the pattern; it matches any character except characters p to z. |
| <code>Xy</code> | Matches x followed by y. |
| <code>x y</code> | Matches either x or y. |

Simboli regular expression riferiti alla parola

| Symbol | Description |
|-----------------|--|
| <code>\d</code> | Matches digits (equivalent to <code>[0-9]</code>). |
| <code>\D</code> | Matches non-digits. |
| <code>\w</code> | Matches word characters. |
| <code>\W</code> | Matches non-word characters. |
| <code>\s</code> | Matches whitespaces (equivalent to <code>[\t\r\f\n]</code>). |
| <code>\S</code> | Matches non-whitespaces. |
| <code>\b</code> | Matches word boundary when outside bracket. Matches backslash when inside bracket. |
| <code>\B</code> | Matches non-word boundary. |
| <code>\A</code> | Matches beginning of string. |
| <code>\Z</code> | Matches end of string. |

Regular expression Quantificatori

| Symbol | Description |
|------------|---|
| expr? | Matches 0 or 1 occurrence of expr (equivalent to expr{0,1}). |
| expr* | Matches 0 or more occurrences of expr (equivalent to expr{0,}). |
| expr+ | Matches 1 or more occurrences of expr (equivalent to expr{1,}). |
| expr{x} | Matches x occurrences of expr. |
| expr{x, y} | Matches between x and y occurrences of expr. |
| expr{x,} | Matches x or more occurrences of expr. |

Pattern non valido

Se il pattern non ha una sintassi corretta di espressione regolare allora a **RUNTIME** si ha un'eccezione di **java.util.PatternSyntaxException**

Java supporto ai file (package NIO)

Classe Console

La console si riferisce alla lettura di dati da tastiera e la sua visualizzazione sullo schermo. Il riferimento alla Console si può ottenere utilizzando il metodo `System.console()`; il quale ritorna null se la JVM non ha alcuna console associata.

I metodi principali della classe Console

| | |
|--|---|
| Reader <code>Reader()</code> | Ritorna un oggetto reader per la lettura da console |
| PrintWriter <code>writer()</code> | Ritorna un oggetto <code>PrintWriter</code> per stampare a video. |
| String <code>readLine()</code> | Legge una riga dalla console e ritorna null se per esempio premiamo Ctrl+Z o Ctrl+D |
| String <code>readLine(String fmt, Object...args)</code> | Come <code>readLine</code> ma con un prompt |
| Void <code>Flush()</code> | Forza la stampa del buffer restante della console |
| Char[] <code>readPassword()</code> | legge una password |
| Char[] <code>readPassword(String fmt, Object....args)</code> | Legge una password con un prompt |
| Console <code>format(String fmt, Object...args)</code> | Stabilisce un formato per il prompt |
| Console <code>printf(String fmt, Object...args)</code> | Stampa a video un prompt |

Perché Char[] password

Qualcuno potrebbe fare il Dump della memoria heap, potendo leggere il contenuto dello Heap (nello Stack ci sono solo indirizzi di memoria che puntano agli oggetti presenti nello Heap ed è non è leggibile).

Quindi per motivi di sicurezza si tiene la password su una variabile primitiva char perché risiede sullo stack perché una stringa è leggibile dalla memoria heap.

Il package NIO

Il package NIO contiene 4 classi astratte :

| | |
|--|---|
| File di testo , contiene caratteri leggibili da qualsiasi editor di testo | File binari , contiene oggetti leggibili dalle specifiche applicazioni |
| Reader
Writer | InputStream
OutputStream |
| Classi concrete | Classi concrete |
| UnBuffered
FileReader
FileWriter | UnBuffered
FileInputStream
FileOutputStream |
| Buffered
BufferedReader(FileReader)
BufferedWriter(FileWriter)
Metodi aggiunti
readLine()
WriteLine();
reset(); svuota buffer e si posiziona all'inizio del file | Buffered
BufferedReader(FileInputStream)
BufferedWriter(FileOutputStream) |

Metodo read()

Il metodo read() si occupa di leggere il contenuto di un file e nel caso di lettura di un file di testo ritorna -1 se non ci sono più caratteri da leggere.

Metodo ReadLine() solo verso buffered

Legge un'intera linea di testo e se la linea è nulla allora il testo è finito.

try with resources

L'apertura di un file lancia eccezioni di tipo checked che siamo obbligati a gestire. In particolare per i file si usa il blocco try with resources perché tutte le classi che gestiscono i file implementano l'autocloseable.

L'interfaccia autocloseable

L'interfaccia autocloseable obbliga le classi che la implementano ad avere il metodo close per la chiusura della risorsa, nel nostro caso del file. Nel caso di classi con buffer nel metodo close richiama anche il metodo flush() che assicura la completa lettura e scrittura del buffer.

Operazione su file di testo

Su un file di testo ci sono operazioni comuni che spesso facciamo:

- cercare una stringa
- cercare un pattern
- sostituire una sequenza di caratteri

La classe Scanner

La classe Scanner ci permette di fare tutte queste operazioni trattando un file di testo alla stessa stregua di una stringa di testo con metodi simili a quelli già visti per le stringhe.

```
Scanner sc=new Scanner(new FileReader(nomefile));
```

File binary (stream di byte)

Tutti i file binari hanno una caratteristica in comune, i primi byte del file binario (scritti però in esadecimale) identificano il formato del file (numero magico).

"0xCAFEBAE" → questo è un file .class

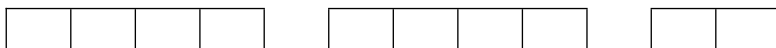
I Magic number sono dei valori esadecimali noti per ogni tipo di file, il numero di byte utilizzati per il magic number è variabile.

Metodo read() di n byte

Il metodo read() si occupa di leggere n byte (dimensione di un array) da un file binario e se non ci sono più byte da leggere ritorna -1.

Byte[] b=new Byte[4]; → array di 4 byte

Read(b); → legge 4 byte alla volta e li memorizza in b



Leggiamo i primi 4 byte, i 4 byte successivi, ed infine i restanti byte che sono 2.

Metodo read() di tipi primitive (es. int, char, short, long, float, double)

Se il nostro file contiene vari tipi di dati: prima un int, poi short, poi un double); ogni volta dobbiamo ricordare di leggere n byte, 4 per l'int e n byte per un altro tipo di dato.

L'introduzione di questi metodi permette di leggere il dato specifico.

| DataInputStream | DataOutputStream |
|------------------------|-------------------------|
|------------------------|-------------------------|

- | | |
|-------------|--------------|
| - readChar | - writeChar |
| - readInt | - writeInt |
| - readShort | - writeShort |

File di Oggetti (serialization)

Tutto il contenuto di un oggetto viene scritto su un file. Il processo di conversione di un oggetto in memoria in una sequenza di byte è nota come serializzazione.

| ObjectInputStream | ObjectOutputStream |
|--------------------------|---------------------------|
|--------------------------|---------------------------|

- | | |
|----------------|-----------------|
| - readObject() | - writeObject() |
|----------------|-----------------|

ClassNotFoundException

Di solito le eccezioni lanciate nell'apertura di un file sono tutte checked e sono:

- FileNotFoundException
- IOException

Se dobbiamo leggere un oggetto da un file allora potrebbe capitare che l'oggetto letto non corrisponde con l'oggetto in memoria lanciando un'eccezione di tipo ClassNotFoundException.

SerialVersionUID=1L è la soluzione al ClassNotFoundException; conserva un valore progressivo che tiene traccia delle modifiche dell'oggetto.

Serialization

Una classe non è serializzabile di default. Quindi un'oggetto di default non può essere scritto; deve implementare l'interfaccia Serializable per farlo. Possiamo inoltre decidere di non serializzare tutti gli attributi del nostro oggetto dichiarandoli con la keyword "transient".(variabile transitoria che non sarà salvata permanentemente).

Il valore dell'attributo non sarà salvato, in memoria nel file sarà salvato il valore di default del tipo di attributo considerato.

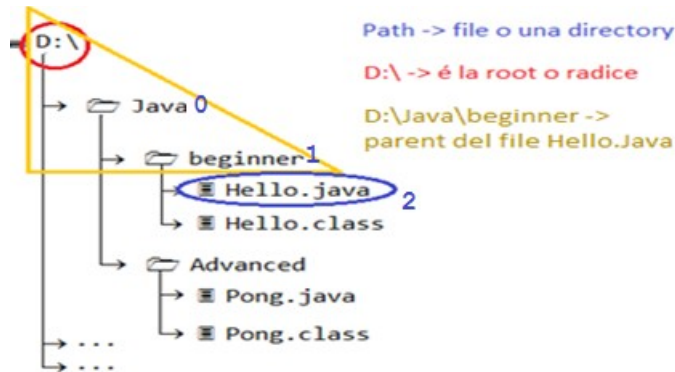
Se un oggetto contiene un oggetto non serializzato (es.Punto); è inutile tentare di scrivere o leggerlo, la keyword transient mi escluderà l'oggetto punto non scrivibile.

```
classe Punto { }  
classe Cerchio implements Serializable{  
    Punto transient p  
}
```

Java supporto ai file (package NIO2)

Classe Path implementa Comparable, Iterable e Watchable

Rappresenta un path nel nostro file di sistema. Il file di sistema ha una struttura ad albero dove ogni ramo rappresenta un path di indice i ; l'indice parte da 0 dal nodo immediatamente successivo alla radice (D:\) e finisce con il file o la directory interessata (getFileNames).



Path assoluto -> parte dalla radice

Path relativo -> da un nodo ad un altro nodo

subPath(0,3)=Java\beginner\
Hello.java -> porzione di path

Metodi dell'interfaccia Path

| | |
|--|--|
| Path getRoot() | Recupera la radice da un path e ritorna nulla se il path non ha radice |
| Path getFileName() | Recupera il nome del nodo finale |
| Path getParent() | Recupera il percorso del path |
| int getNameCount() | Conta il numero di nodi attraversati |
| Path getName(int index) | Recupera il nome del nodo i -esimo |
| Path subpath(int beginIndex, int endIndex) | Recupera una porzione di path dato due indici, come subString(). Questo metodo può lanciare l'eccezione IllegalArgumentException se come argomenti inseriamo indici errati |
| Path normalize() | Elimina nodi di path non necessari.
Es. ".\test" = a test |
| Path resolve(Path other)
Path resolve(String other) | Ritorna il path per andare da un ramo ad un altro del file di sistema |
| Boolean isAbsolute() | Questo metodo ritorna vero se il path é un path assoluto e falso se é un path relativo |
| Path startsWith(String path)
Path startsWith(Path path) | Ritorna vera se il path inizia con la stringa o il path passato come argomento |
| Path toAbsolutePath() | Se il path é già assoluto ritorna se stesso, altrimenti recupera il path assoluto attraverso il file system |
| Path toRealPath | Recupera un path reale partendo da un path e lancia noSuchFileException se il path non esiste nel nostro file system |

toPath package NIO2 (Java 7) per passare al package NIO
toFile package NIO invece per passare al package NIO2

Comparing di due path

I metodi `Equals()` e `CompareTo` sono stati implementati nella classe `path` perciò è possibile confrontare due path come se fossero due stringhe.

I path confrontati però devono essere assoluti e normalizzati.

Classe Files

Classe di Utility per fare operazioni sui file e le directory (sono final con costruttore privato che non può essere istanziato e tutti i metodi sono statici)

| | |
|---|---|
| <code>Path createDirectory(Path dirPath, FileAttribute<?>... dirAttrs)</code>
<code>Path createDirectories(Path dir, FileAttribute<?>... attrs)</code> | Crea una directory e ne restituisce il path.
Crea più directory intermedie per ricostruire l'intero percorso |
| <code>Path createTempFile(Path dir, String prefix, String suffix, FileAttribute<?>... attrs)</code> | Crea un file temporaneo con un numero progressive e lo ritorna in un path |
| <code>Path createTempDirectory(Path dir, String prefix, FileAttribute<?>... attrs)</code> | Come il precedente ma una directory temporanea |
| <code>Path copy(Path source, Path target, CopyOption... options)</code> | Copia il file da una sorgente ad una destinazione. <code>CopyOption</code> sono <code>REPLACE_EXISTING</code> , <code>COPY_ATTRIBUTES</code> , o <code>NOFOLLOW_LINKS</code> . Questo metodo può lanciare <code>FileAlreadyExistsException</code> . |
| <code>Path move(Path source, Path target, CopyOption... options)</code> | Simile al precedente ma cancella il sorgente |
| <code>boolean isSameFile(Path path, Path path2)</code> | Verifica se i path di due file sono uguali (nomi di directory e file uguali) |
| <code>boolean exists(Path path, LinkOption... options)</code> | Se un file o una directory esiste |
| <code>Boolean isRegularFile(Path path, LinkOption...)</code> | Se il path è un file (il contrario di <code>isDirectory()</code>) |
| <code>Boolean isSymbolicLink(Path path)</code> | Se il file su cui ci troviamo è un collegamento ad un altro file |
| <code>Boolean isHidden(Path path)</code> | Se il file o la directory è nascosta |
| <code>long size(Path path)</code> | La dimensione del file |
| <code>UserPrincipal getOwner(Path path, LinkOption...), Path setOwner(Path path, UserPrincipal owner)</code> | Il nome utente, proprietario del file |
| <code>FileTime getLastModifiedTime(Path path, LinkOption...), Path setLastModifiedTime(Path path, FileTime time)</code> | Data e ora dell'ultima modifica fatta ad un file o una directory |
| <code>Object getAttribute(Path path, String attribute, LinkOption...), Path setAttribute(Path path, String attribute, Object value, LinkOption...)</code> | Recupera alcuni attributi su file o directory quali: data creazione, dimensione, ecc..... |

Walking a file tree

E' necessario percorrere un intero percorso del nostro file di sistema:

- per cercare un file o directory
- per copiare un'intera directory (file e sottodirectory con file)

L'unico argomento necessario per esaminare il nostro sottoalbero è il percorso di partenza.

Per attraversare il nostro sottoalbero java mette a disposizione due metodi:

- `Path walkFileTree(Path start, FileVisitor<? Super Path> visitor)`
- `Path walkFileTree(Path start, Set<FileVisitOption> options, intMaxDepth, FileVisitor<? Super Path> visitor)`

`intMaxDepth` → massima profondità dell'albero da esplorare (n nodi)

Classe che implementa interfaccia `FileVisitor<? Super Path> visitor`

Ad ogni nodo che attraverso l'interfaccia `FileVisitor` obbligherà la nostra classe ad implementare. i metodi che ci diranno cosa fare se il nodo attraversato o da attraversare è un file oppure una directory.

| | |
|--|--|
| <code>FileVisitResult preVisitDirectory(T dir, BasicFileAttributes attrs)</code> | Metodo invocato quando il nodo da visitare è una directory |
| <code>FileVisitResult visitFile(T file, BasicFileAttributes attrs)</code> | Metodo invocato quando il nodo da visitare è un file |
| <code>FileVisitResult postVisitDirectory(T dir, IOException exc)</code> | Metodo invocato quando usciamo da una directory |
| <code>FileVisitResult visitFileFailed(T file, IOException exc)</code> | Metodo invocato quando fallisce qualcosa |

La classe `SimpleFileVisitor` contiene già tutti questi metodi vuoti. E' comoda perché non siamo obbligati ad implementare tutti i 4 metodi.

Questi metodi restituiscono un `FileVisitResult`, un valore enumerativa che ci indica cosa fare ad ogni nodo:

`FileVisitResult` enum

`CONTINUE` : continua a visitare i nodi successivi

`TERMINATE` : Termina di visitare il percorso

`SKIP_SUBTREE` : salta il resto del sottoalbero

`SKIP_SIBLINGS` :salta tutti i nodi allo stesso livello

Es. l'elemento corrente è un file

Si entra nel metodo `visitFile` stampiamo il nome del file corrente

E dopo diciamo di continuare a visitare l'albero (`FileVisitResult.CONTINUE`).

PathMatcher

La classe `PathMatcher` è usata per cercare un file o directory nel nostro file system utilizzando le regular expression.

Pattern → glob:*.exe (la stringa di pattern deve iniziare con glob:)

Watching a directory (WatchService)

Un osservatore che guarda una directory ci avvisa quando c'è un cambiamento :

- Modifiche
- Creazioni
- Cancellazioni

La classe WatchService crea un osservatore al quale dire per esempio che deve osservare (register) solo le modifiche dentro uno specifico Path.

Ciclo infinito (;;;) per avere un osservatore (watchEvent) a ciclo infinito sul path e vedere se un evento di tipo modifica di è verificato:

```
( for (WatchEvent<?> event : key.pollEvents()) ) tutti gli eventi  
Switch (event) per dire cosa fare quando l'evento che ci interessa si è verificato.
```

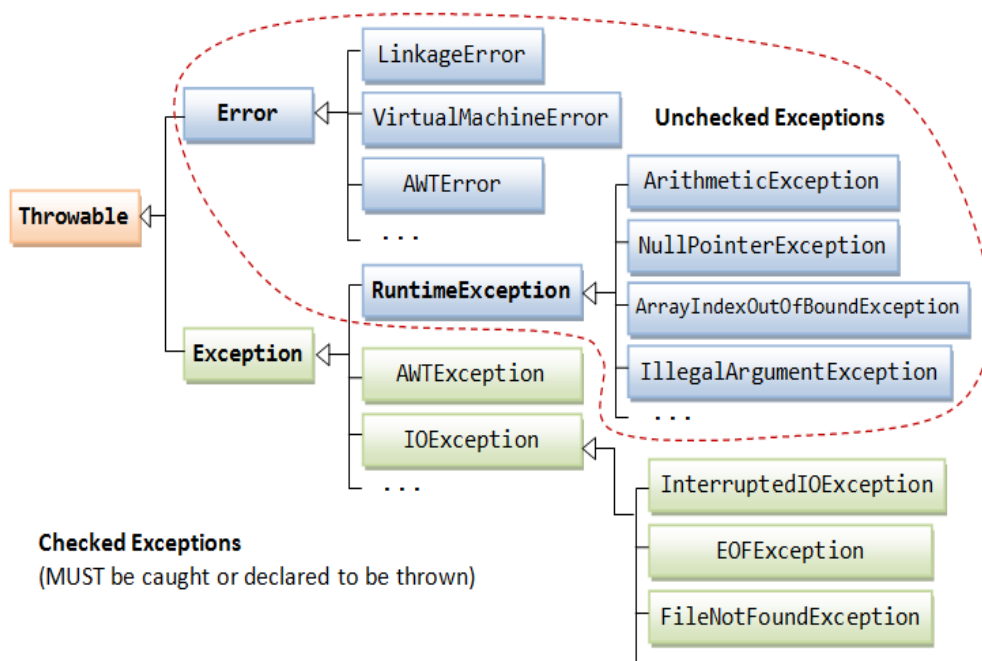
ES.

- ENTRY_MODIFY
- OVERFLOW

Eccezioni e asserzioni

Possiamo trattare come eccezioni tutti gli oggetti che sono in relazione di ereditarietà con la classe throwable o sue sottoclassi.

La cattura di un'eccezione genera un'istanza dell'eccezione catturata. Se ad esempio il nostro codice solleva un'eccezione di tipo ArithmeticException e nel catch viene catturata, creerà un'istanza della classe ArithmeticException.



Tipi di eccezioni

- **Errori:** Sono eccezioni per le quali non ha senso continuare con l'esecuzione del programma ed è inutile tentare di catturarle e gestirle. Di solito sono causate da interruzioni del sistema o della JVM.
- **Checked :** Questo tipo di eccezioni siamo obbligati a gestirle e sono causate da eventi esterni imprevedibili. Queste si verificano in caso di malfunzionamento di una risorsa che determinano per esempio la lettura fallita di un file o la perdita di connessione al database.
- **Unchecked :** Questo tipo di eccezioni non siamo obbligati a gestirle ma è una buona pratica evitare che il nostro codice sollevi una unchecked. Queste eccezione sono causate da inserimenti errati dell'utente o da errori di programmazione.

A runtime (unchecked exception) non controllate

Sono eccezione prevedibili, lanciate a causa di errori da parte dell'utente o del programmatore e proprio perché prevedibili devono essere evitate facendo più controlli nel codice.

Sono dette non controllate, perché non siamo obbligati a gestirle

A compiletime (checked exception) controllate

Queste eccezioni non sono prevedibili ma il compilatore ci aiuta a individuarle, obbligandoci a gestirle (throws e try catch) altrimenti non compila; per questo motivo sono dette controllate.

Tutte le classi Java hanno metodi che lanciano eccezioni al chiamante:

- se sono di tipo unchecked possiamo anche non gestirle e java ci stamperà a video lo stacktracer delle eccezioni lanciate e non gestite,
- se invece sono di tipo checked dobbiamo gestirle.

Modalità di gestione delle eccezione

- Cattura l'eccezione (try catch)
- Rimanda l'eccezione al chiamante (throws) che a sua volta la ripassa al chiamante o la gestisce con il (try catch)
- Cattura l'eccezione e nel catch rilancio l'eccezione al chiamante (throw)

Stacktrace

Se l'eccezione non viene intercettata e gestita dove si verifica, si propaga attraverso la gerarchia delle chiamate di metodi (ovvero lo stack dei record di attivazione)

Nello stack tracer vengono chiusi tutti i record di attivazione fino ad incontrare un blocco try ... catch che gestisca l'eccezione.

Catch multiplo e il multi-catch

| Catch multiplo -> eccezioni in ereditarietà | Multi Catch -> eccezioni non inereditarietà |
|--|---|
| <p>Per gestire più eccezione separatamente</p> <pre>Try {
 Statements
}
Catch (EccezioneGestita1 e1) {
 Statements
}
Catch (EccezioneGestita2 e2) {
 Statements
}</pre> <p>Se le eccezioni sono in ereditarietà tra di loro bisogna mettere le eccezioni più generiche alla fine.</p> | <p>Per gestire più eccezioni allo stesso modo</p> <pre>Try {
 Statements
}
Catch (EccezioneGestita1
EccezioneGestita2 e) {
 Statements
}</pre> <p>Le eccezioni non devono essere in gerarchia di ereditarietà (es. classe padre/figlia)</p> |

Eccezioni concatenate

Le eccezioni concatenate si adoperano quando vogliamo catturare un'eccezione e all'interno della stessa sollevarne un'altra da rilanciare al chiamante. In questo modo possiamo associare un'eccezione ad un'altra che è stata la causa che ha generato questa eccezione rilanciata.

L'eccezione causa può essere settata una sola volta.

Come si rilancia un'eccezione al chiamante

Throw new exception(throwable cause) → come parametro abbiamo un'altra eccezione che è stata la causa che ha sollevato questa eccezione.

Throw new exception(String DetailMsg, throwable cause) → uguale alla precedente ma c'è il parametro. DetailMsg passato al costruttore

getCause() → recupera la causa (oggetto Throwable) che ha generato l'eccezione per rilanciarla al chiamante.

initCause() → setta la causa che ha generato l'eccezione da rilanciare.

Due modi per creare un'eccezione concatenata (Chained exception)

| | |
|---|---|
| <pre>Throw new exception(throwable
cause)

try {
 statements
}
catch (exception e) {
 ...statements</pre> | <pre>InitCause() setta la causa che lancia
l'eccezione

try {
 statements
}
catch (exception e) {
 ...statements
 Throwable eccezione= new exception ();</pre> |
|---|---|

```
...throw new exception (e);  
}
```

```
eccezione.InitCause(e);  
throw eccezione;  
}
```

IllegalStateException (non posso settare due volte la causa)

```
Exception eccezione=throw new exception (e);  
eccezione.InitCause(e);
```

```
Exception eccezione=throw new exception ();  
eccezione.InitCause(e);  
eccezione.InitCause(e);
```

Blocco Finally

Il garbage collector elimina gli oggetti presenti nello heap che non trovano un riferimento attivo nello stack. Questo vale però per gli oggetti presenti in memoria ma non per i file e le connessioni database, che restano attive finché non siamo noi a rilasciare le risorse, chiudendole (ciò non vale per il try with resources che rilascia la risorsa alla chiusura del try perché implementano implicitamente l'autocloseable).

L'interfaccia closeable obbliga le classi di gestione di file e di connessioni database ad implementare il metodo close.

Il blocco finally è il posto ideale dove richiamare il metodo close perché questo blocco viene sempre e cmq eseguito dopo il try catch.

Solo con la system.exit(-1) il blocco finally non viene eseguito ma è considerata una bad practice proprio perché non ci garantisce la chiusura delle risorse (file o connessioni aperte).

N.B. Nel blocco finally vanno rilasciate le risorse e non vanno inserite istruzioni che modificano il flusso di esecuzione delle istruzioni come ad esempio (return).

Throw

Posso lanciare sia checked che unchecked exception, anche se nel caso di una unchecked non siamo obbligati (ma possiamo farlo) a lanciarla al chiamante (throws nella firma del metodo).

Checked

```
void pippo() throws IOException {  
    Throw new IOException();  
}
```

Unchecked

```
void pippo(){  
    Throw new NullPointerException();  
}
```

N.B. in una classe figlia oltre a gestire o non gestire l'eccezione nel metodo in override della classe padre possiamo aggiungere tutte le eccezioni di tipo Unchecked che vogliamo.

Rethrow exception (Java 7)

```
class PreciseRethrow {
    public static void main(String[] str) {
        try {
            foo();
        } catch (NumberFormatException ife) {
            System.out.println(ife);
        }
    }
    static private void foo() throws NumberFormatException, IOException {
        try {
            int i = Integer.parseInt("ten");
            Path p = paths.get("");
            p.toRealPath(null);
        } catch (Exception e) {
            throw e;
        }
    }
}
```

parseInt("ten") → l'unica eccezione che può sollevare è la **NumberFormatException**

Allora anche se **catturiamo una Exception staticamente** possiamo dire al chiamante che sarà sufficiente gestire una **NumberFormatException**.

N.B. Possiamo anche avvisare il chiamante che ci sono due eccezioni da gestire. Ecc...

Try with Resources (Java 7)

Durante la dicitura del nostro codice è possibile dimenticare di chiudere le nostre risorse nel blocco finally. Con questo nuovo fattore introdotto in java 7 le risorse aperte nel try saranno automaticamente chiuse alla fine del blocco try catch.

Questo è possibile per tutte le risorse che implementano l'interfaccia auto-closeable.

```
Try (istanziare la risorsa);(istanziare la risorsa) {
    // Risorse → Non posso reistanziare i nostri oggetti risorsa (new),
    Statements
}
Catch (exception e) {
    Statements
    // qui la risorsa non è visibile
}
```

N.B. La visibilità della risorsa è limitata al blocco try.

Eccezioni all'interno di blocchi statici e non statici

- I blocchi statici non possono lanciare eccezione di tipo checked.

```
public class Example {
    int i=0;
    static {
        if (i==0)
            throw new IOException();
    }
}
```

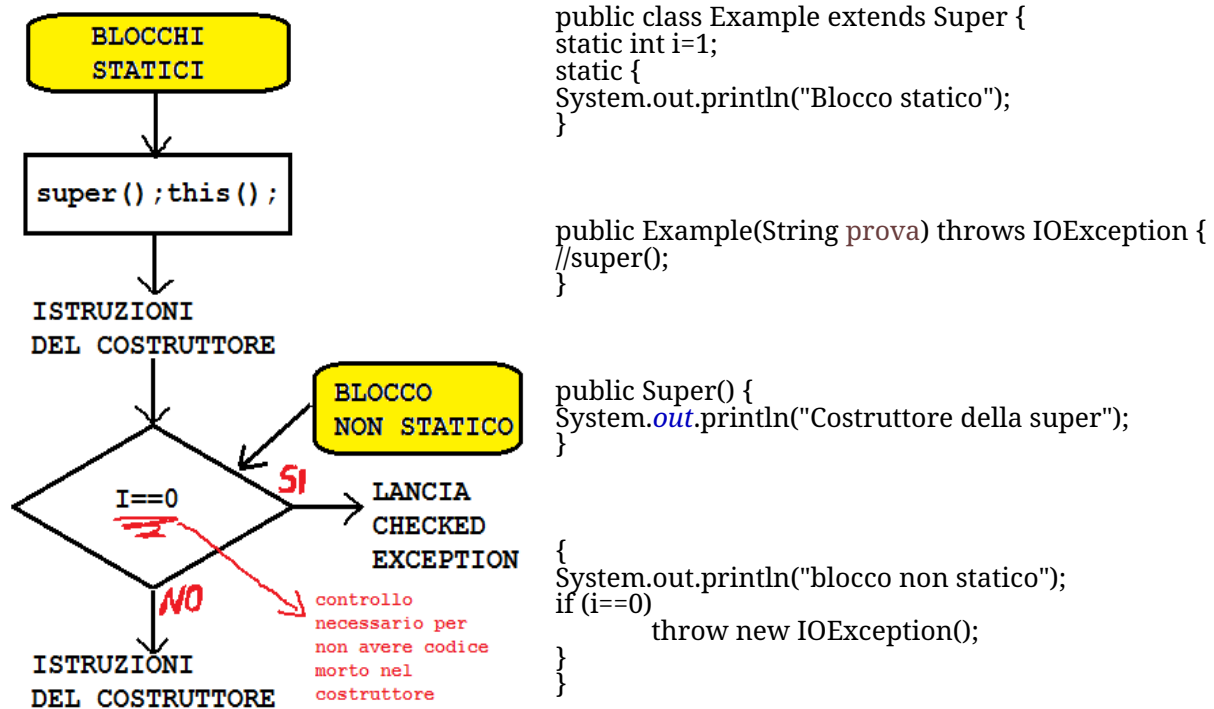
```
//il compilatore avvisa che dentro ad un blocco statico l'eccezione di tipo checked può solo essere gestita ma non lanciata  
}  
}
```

- I blocchi non statici possono lanciare eccezioni di tipo checked ma tutti i costruttori devono dichiararli nella loro firma. Questo perché il compilatore unisce il costruttore a tutto il codice contenuto nel blocco non statico.

```
Public class Example {  
    int i=0;  
    {  
        if (i==0)  
            throw new IOException();  
        //il compilatore mi avvisa che posso gestirla solo con il try catch  
    }  
  
    Public Example() throws IOException {  
    }  
}
```

N.B. il blocco non statico viene gestito dopo la chiamata al super o al this.

Flusso di esecuzione di un programma java



Custom Exception

Noi possiamo decidere di creare delle nostre eccezioni:

- di tipo checked estendendo la classe `Exception`
- di tipo Unchecked estendendo la classe `RuntimeException`

Una nostra eccezione viene implementata quando oltre alla visualizzazione del classico messaggio si vuole fare altre cose, esempio scrivere un file di log, mandare un email, ecc...

Assertion

Spesso diamo per scontato che una determinata condizione in un punto del codice sia vera.

Ma se noi volessimo essere davvero certi che quanto dato per scontato sia proprio vero? questo si traduce in un'asserzione.

Le asserzioni sono codici aggiuntivi che consentono:

- di interrompere l'esecuzione del programma con il lancio di un `AssertionError`, che non va gestita, se una data condizione non si verifica indicandoci tempestivamente il punto in cui qualcosa è andato storto.
- di dare ad altri programmatori più leggibilità del codice, i programmatori sapranno grazie alle assertion cosa abbiamo dato per scontato nel nostro codice.

Le assertion occorrono solo nella fase di debug e testing del nostro programma, vanno perciò abilitate nell'esecuzione (`-ea`). Nell'applicativo finito, il codice ha tutte le assertion verificate; però potrebbe comunque presentare eccezioni e potrebbero essere necessari dei controlli che vanno implementati con tutto quello studiato finora.

N.B. La condizione booleana dell'assertion non deve fare nulla che modifichi l'esecuzione del programma quando le disabilitiamo.

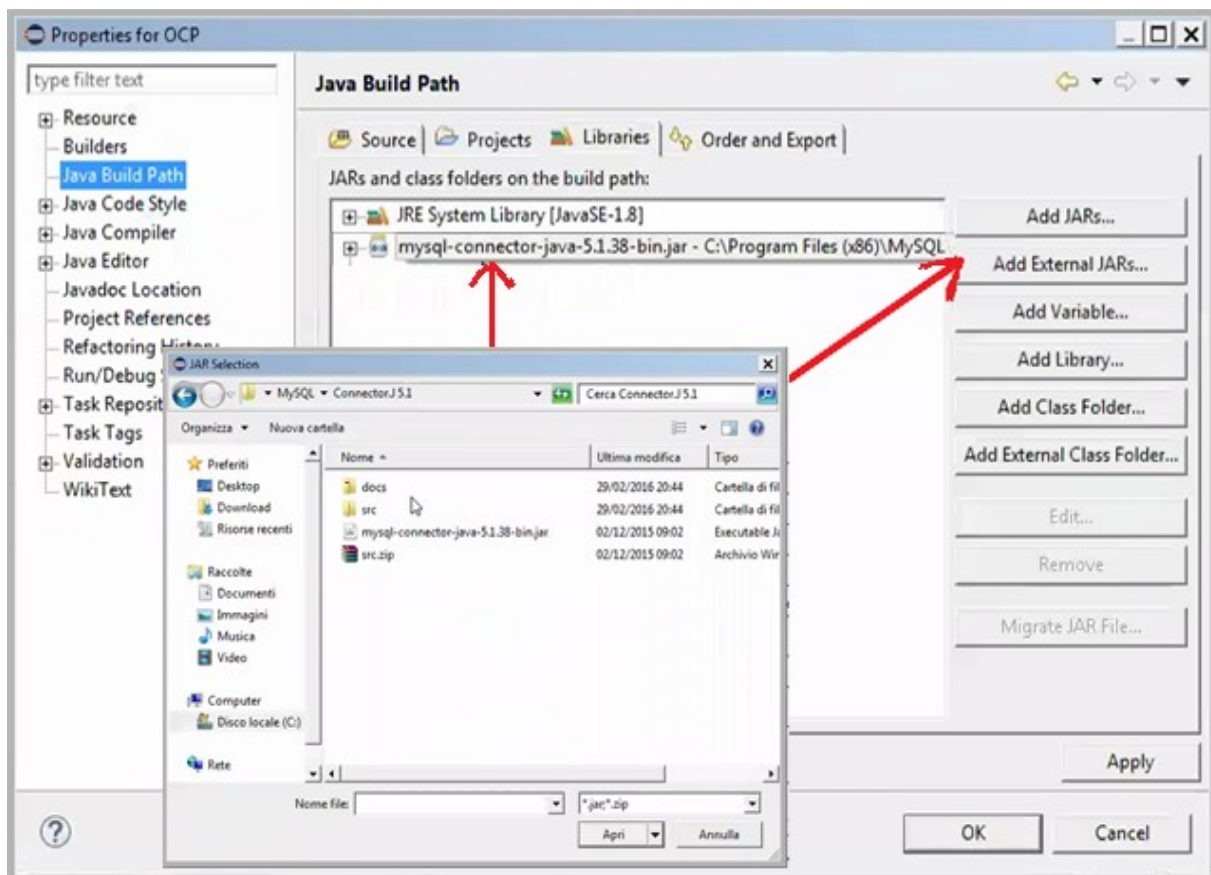
Building database applications with JDBC

Java database connectivity è una importante libreria di Java (API) contenuta nel package `javax.sql`, che fornisce delle funzionalità di accesso e di manipolazione (inserimenti, aggiornamenti e creazioni) di un qualsiasi tipo di database.

Noi abbiamo diversi tipi di database (Mysql, Oracle, DB2, SQLServer, ecc...) e il JDBC ci permette di poter sfruttare il nostro programma con tutti i diversi database modificando semplicemente la stringa di connessione e caricando il driver JDBC del tipo di database utilizzato.

Aggiungiamo un database nel nostro progetto

Una volta installato il database ed avviato il servizio del database



Aggiungiamo la jar del database che ci interessa nel nostro progetto.

Una volta eseguiti questi passaggi possiamo richiamare il database aggiunto usando una semplice stringa di connessione.

```

class DbConnector {
    private final static String url = "jdbc:mysql://localhost:3306/";
    private final static String database = "addressBook";
    private final static String userName = "root";
    private final static String password = "mysql123";

    static Connection connectToDb() throws SQLException {
        return DriverManager.getConnection(url + database, userName, password);
    }
}

```

La stringa di connessione include un url+nome del database, l'utente e la password...

Il posto ideale in cui tentare la connessione al database è nel blocco **try with resources** per essere certi che la connessione una volta che non sarà più necessaria venga automaticamente chiusa (autocloseable).

N.B. I record del database non sono memorizzati in modo ordinato e se vogliamo visualizzare i record in un ordine preciso è opportuno specificarlo con un orderby

Interrogazioni del database (query)

Le interrogazioni sul database sono dette query. Le query possono essere: SELECT, INSERT, UPDATE, DELETE

Le SELECT ci consentono di estrarre dati dal database

Le INSERT ci consentono di inserire dati nel database

Gli UPDATE ci consentono di aggiornare i dati

Le DELETE ci consentono di eliminare i dati

IL JDBC fornisce due interfacce per fare le queries : Statement e ResultSet.

Statement

Sono le istruzioni query abitualmente utilizzate per interrogare un database. Ci sono 3 tipi di statment:

- 1) Statement(query pura) → senza alcun parametro che è rappresentato dal metodo CreateStatement()
- 2) PreparedStatement (query con parametri) → che è rappresentato dal metodo prepareStatement()
- 3) CallableStatement (Esecuzione delle stored procedures) → sono procedure del database che possiamo richiamare tramite il metodo prepareCall().

Una volta creato l'oggetto statement possiamo eseguire la nostra query con i metodi executeQuery(), executeUpdate, o il deprecato metodo execute().

- ExecuteQuery() ritorna un resultSet contenente i record estratti dalla select
- ExecuteUpdate() ritorna un intero rappresentante il totale dei record inseriti o aggiornati o cancellati

- `Execute()` essendo la combinazione dei precedenti ritorna o `ResultSet` o interi e con i metodi `getResultSet()`, `getUpdateCount()` recuperiamo i valori; con `getMoreResults()` possiamo sapere quale tipo di query sarà la prossima.

ResultSet

Il `resultSet` è quello che ci viene restituito dal database dopo una query, il `resultSet` è un insieme di dati in formato tabellare (righe e colonne) dove un cursore punta ad un record di questa tabella.

| | NOME | COGNOME | TELEFONO |
|---|------|---------|----------|
| 1 | | | |
| 2 | | | |
| 3 | | | |
| 4 | | | |

I metodi del `resultSet` ci consentono di scorrere la tabella:

- lungo le righe, mediante un cursore che ci consente di posizionarci sul record desiderato,
- lungo le colonne, per ottenere (`getDouble`, `getString`) all'interno di un record il campo del tipo di dato desiderato. Un campo può essere recuperato:
 - tramite un indice successivo che parte da 1, (`getDouble(int numeroColonna)`)
 - tramite il nome del campo stesso. (`getDouble(String nomeColonna)`).

N.B. E' importante che il tipo di dato recuperato con il `get` concordi con il tipo di dato del campo del database, ovvero non posso fare un `getInt` di una stringa, però posso tranquillamente recuperare un intero come stringa.

Aggiornare il database

Abbiamo due modi per modificare i dati nel nostro database:

1. Utilizzando uno statement di insert, update o delete
2. Oppure modificando un `resultSet` utilizzando i metodi update.

Per quanto riguarda lo statement

Una volta creato uno statement, sarà sufficiente eseguire il metodo `executeUpdate(stringa)` con il parametro stringa che rappresenta la nostra query.

Esempio

```
try (Connection connection = DbConnector.connectToDb();
    Statement statement = connection.createStatement();
```

```

{
String sql="INSERT INTO CONTACT (firstName, email)"
        + " VALUES ('Pluto','pluto@libero.it)";
int result=statement.executeUpdate(sql);
}
catch (Exception e) {
    e.printStackTrace();
    System.exit(-1);
}

```

```

NB la creazione di una tabella si fa solo tramite query : int result
= statement.executeUpdate("CREATE TABLE familyGroup (id int not null
auto_increment, nickName varchar(30) not null, primary key(id));");

System.out.println("Table created successfully");

```

Per quanto riguarda il resultSet

Un volta recuperato un resultSet da uno statement è necessario specificare nella creazione dello statement che il resultSet è modificabile.

```

Statement statement = connection.createStatement(ResultSet.TYPE_SCROLL_SENSITIVE,
    ResultSet.CONCUR_UPDATABLE);
ResultSet resultSet = statement
    .executeQuery("SELECT firstName, email "
        + "FROM contact WHERE firstName=\"Michael\"")

```

Ora possiamo modificare un record

```

resultSet.absolute(1);//cursore sul record da modificare
resultSet.updateString("phoneNo", "+919976543210");
// reflect those changes back to the database by calling updateRow() method
resultSet.updateRow();

```

inserire un record

```

resultSet.moveToInsertRow(); //cursore su un nuovo record vuoto
resultSet.updateString("firstName", "John");
resultSet.updateString("lastName", "K.");
resultSet.updateString("email", "john@abc.com");
resultSet.updateString("phoneNo", "+19753186420");
resultSet.insertRow();

```

o cancellare un record

```

if(resultSet1.next()){
// delete the first row
resultSet1.deleteRow();
}
resultSet1.close();

```

ResultSet.updateRow(), resultSet.insertRow(), resultSet1.deleteRow();

Questi metodi confermano la modifica del resultSet, finché non viene fatta questa conferma il resultSet visualizzerà i vecchi valori.

A cosa servono? Ci consentono di ritornare allo stato precedente del resultSet con il metodo **resultSet.cancelRowUpdates()**.

Metadata di un database

I metadata di un database sono le informazioni sulla struttura del database:

- o Numero massimo di colonne di una tabella
- o Numero totale delle tabelle
- o Nomi delle tabelle
- o I campi di una tabella
- o Driver db utilizzato
- o Relazioni fra tabelle
- o Ecc.. ecc..

```
DatabaseMetaData metaData = connection.getMetaData();
```

Transazioni

La transazione è insieme di query che devono andare tutte a buon fine. Un esempio classico è la transazione di una banca che è formata da due query:

1. inserire una riga con importo negativo sul conto del committente
2. Inserire una riga con importo positivo sul conto del destinatario

Queste due sql devono entrambe andare a buon fine per determinare che l'operazione di transazione bancaria si è conclusa con successo. (stato coerente se entrambe vengono fatte, altrimenti è stato incoerente)

ACID

Per garantire il successo di una transazione il database deve soddisfare le proprietà Atomicity Consistency Isolation Durability:

- Atomicity :tutte le sql devono andare a buon fine
- Consistency :il database è in stato coerente sia prima che dopo
- Isolation :ogni transazione è indipendente da altre transazioni
- Durability :una volta conclusa la transazione i dati saranno persistenti sul database.

NB. Se non diciamo al database di usare transazioni, ogni statement rappresenterà una transazione e i dati verranno immediatamente scritti in modo persistente sul database (subito dopo il metodo updaterow() per esempio).

SetAutoCommit()

Possiamo dire al nostro oggetto connection di :

- committare in automatico le transazioni(true), non gestendo la transazione, essa sarà scritta in automatico sul database dopo il metodo updaterow(),
- di non committare in automatico le transazioni(false), saremo noi a gestire la transazione e decidere quando committare (metodo commit()) ovvero quando rendere persistenti i dati sul database.

RollBack()

E' una buona pratica fare sempre una rollback di una transazione, tuttavia possiamo evitare di chiamare la rollback quando in una singola transazione prestiamo molta attenzione al codice e committiamo solo quando tutte le sql sono realmente andate a buon fine.

E' necessaria se :

- la connessione fa parte di un pool di connessioni (commit di statement pendenti)
- Nell'utilizzo di savepoint

SavePoint()

Possiamo dividere la transazione in più parti e committare fino al savePoint() in tal modo in caso di problemi possiamo

- fare il rollback solo degli statement committati fino a quel momento (savePoint()),
- procedere con la transazione dal punto salvato senza dover riprendere tutta la transazione dal principio.

Interfaccia RowSet()

L'interfaccia rowSet è un'estensione dell'interfaccia ResultSet e forniscono un supporto alle java Beans.

Connected rowSet, jdbcRowSet: Come il resultSet, ha la necessità di mantenere una connessione attiva con il database

Disconnected rowSet, cachedRowSet: Si connette al database solo per leggere e scrivere, dopo resta disconnesso, e sono il più delle volte utilizzati per inviare dati su una rete.

Localization

La possibilità di scrivere il codice di un programma una sola volta e adattarlo al paese in cui verrà utilizzato si traduce in localization. Cosa accadrà al nostro applicativo? Il nostro applicativo visualizzerà tutti i messaggi di testo nella lingua dell'utente finale e utilizzerà tutte le convenzioni del paese per settare la valuta, il formato di un numero (es. la virgola), delle date e dell'ora.

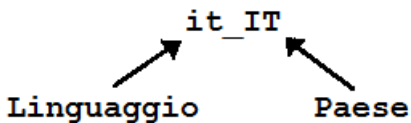
Due importanti linee guide sono da seguire per realizzare un software internazionale:

- non martellare il codice di stringhe, tutte le stringhe andranno scritte su un file di testo separato. (ogni local avrà una chiave (es. `it_IT`) associata ad un file di testo che gestisce la traduzione della stringa per il paese).
- Manipolare aspetti come valuta, numeri, data e ore tenendo conto che possono variare da nazione a nazione.

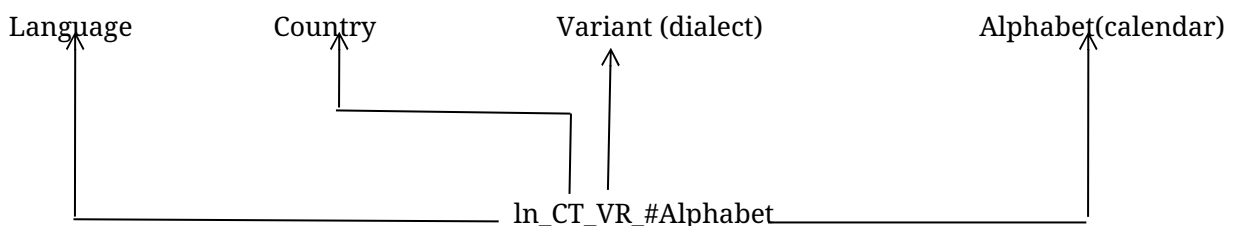
Locales

Un local rappresenta tutte le caratteristiche di una nazione (es. La sua valuta, il suo formato della data, ecc...)

Local più usato

| | |
|---|--|
|  | Il linguaggio definisce la traduzione nella lingua in un paese. Il paese definisce i suoi formati di numeri, valuta, data e ora. Non solo il paese definisce anche il tipo di lingua specifico di un paese (es. l'inglese Inghilterra è diverso dall'inglese degli Stati Uniti). |
|---|--|

Formato di un local



Classe ResourceBundle

E' una classe astratta che si preoccupa:

- **di associare una chiave ad una riga di testo del file di properties** di un specifico local (la classe concreta che fa questo è la `PropertyResourceBundle` che è figlia di `ResourceBundle`),
- **di associare una chiave ad una classe** che è specifica per un local (la classe concreta che fa questo è la `ListResourceBundle` che è figlia di `ResourceBundle`).

Classe ResourceBundle (traduzione di stringhe)

La prima cosa da fare per gestire le nostre stringhe è avere un file di testo (file properties) per ogni lingua e nazione che vogliamo gestire. Se il file di properties non esiste sarà caricato il file di properties di default.

La classe `ResourceBundle` utilizza delle convenzioni sui nomi dei file property in accordo con la local che vogliamo caricare ed ha un nome di file specifico anche per il local di default.

Formato file property

Il nome file deve avere una stessa radice :

(istruzione java)

```
ResourceBundle resBundle = ResourceBundle.getBundle("ResourceBundle",  
                                                    currentLocale);
```

RADICE

← `ResourceBundle.properties` → default

`ResourceBundle_ar.properties` → per la lingua araba

`ResourceBundle_en.properties` → per la lingua inglese

(istruzione java)

```
Locale currentLocale = Locale.ENGLISH;
```

```
System.out.printf(resBundle.getString("Greeting"));
```

file → `ResourceBundle_en.properties`

Dentro questo file troveremo tante chiavi quante sono le stringhe che vogliamo tradurre :

| Chiave(Key Sensitive) | valore |
|-----------------------|---------|
| Greeting | = Hello |

Anzichè stampare `System.out.printf("Hello");`

Stamperemo la chiave `System.out.printf(resBundle.getString("Greeting"));`

la chiave, a runtime, sarà caricata dal corretto file di properties(nel nostro caso nel file inglese) e tradotta nel valore associato(Hello).

Classe ListResourceBundle

Una classe è composta da n attributi che determinano come è fatto un oggetto. Una coppia di elementi di un array bidimensionale può consentirci di associare una chiave ad un attributo del nostro oggetto:

| ATTRIBUTI/CHIAVE | VALORE DELL'ATTRIBUTO |
|------------------|-----------------------|
| MovieName | Che Bella Giornata |
| GrossRevenue | 430000000L |
| Year | 2011 |

Esempio

Consideriamo la classe Movie

```
public class Movie {
    String movieName="";
    Long revenue=0L;
    Integer year = 0;

    Movie(Locale locale) {
        printMovieDetails(ResourceBundle.getBundle("ResBundle", locale));
    }
}
```

Nel costruttore passeremo il locale che ci interessa, e con il metodo printMovieDetails andremo a recuperare il valore degli attributi dalla classe ResourceBundle del locale passato come argomento al costruttore.

```
public void printMovieDetails(ResourceBundle resBundle) {
    movieName = resBundle.getString("MovieName");
    revenue = (Long) (resBundle.getObject("GrossRevenue"));
    year = (Integer) resBundle.getObject("Year");
    System.out.println("Movie " + movieName + "(" + year + ")"
        + " grossed " + revenue);
}
```

Definizione della classe ResourceBundle

```
import java.util.ListResourceBundle;

// Italian version default
public class ResBundle extends ListResourceBundle {
    public Object[][] getContents() {
        return contents;
    }

    static final Object[][] contents = { { "MovieName", "Che Bella Giornata" },
    { "GrossRevenue", (Long) 43000000L }, // in euros
    { "Year", (Integer) 2011 } };
}
```

*N.B. Il metodo **getContents()** è un metodo ereditato da **ListResourceBundle** che deve essere obbligatoriamente implementato, questo metodo restituisce il valore in base alla chiave dall'array **contents**.*

Algoritmo per la scelta del bundle

| | |
|---|--|
| BundleName + "_" + language + "_" + country + "_" + variant | 1. Match esatto tra local e bundle ovvero stessa lingua, paese e variant
Non c'è |
| BundleName + "_" + language + "_" + country + "_" + variant
BundleName + "_" + language + "_" + country
BundleName + "_" + language | 2. Vado a quello man mano più generico eliminando i dettagli del local a destra, quindi cerco prima solo lingua, paese e country eliminando il dettaglio del variant.e così via
Non c'è |

| | |
|---|--|
| BundleName + "_" + defaultLanguage + "_" + defaultCountry
BundleName + "_" + defaultLanguage | 3. Prendo il bundle che ha la lingua e il paese del sistema operativo
Se non esiste vedo se c'è un match tra il bundle e la lingua del sistema operativo
Non c'è |
| BundleName | 4. Prendo il bundle con il locale che noi abbiamo deciso che sia quello di default che non è detto che coincide con il bundle di default del S.O.
Non trova il bundle |
| MissingResourceException | 5. Il nome della radice del bundle non corrisponde e non riesce a trovare il file di properties. |

Metodi della classe ResourceBundle

| Metodo | Descrizione | Lancia eccezione |
|---|--|---|
| Object getObject(String key) | Recupera il valore mappato da una chiave | Lancia l'eccezione MissingResourceException |
| static ResourceBundle getBundle(String radice)

static ResourceBundle getBundle(String radice, Locale locale)

static ResourceBundle getBundle(String radice, Locale locale, Control control) | Recupera un bundle a partire solo da una radice
Carica il bundle di default.
Recupera un bundle a partire da una radice e da un local

Recupera un bundle a partire da una radice e da un local e ogni volta che tenta di trovare un match bundle/local fa quello che è presente nel metodo getCandidateLocales | Lancia l'eccezione MissingResourceException |
| String getString(String Key) | Questo metodo è simile a getObject() ma è specifico per il recupero di stringhe | Oltre a lanciare MissingResourceException quando non trova la key; lancia il ClassCastException se il valore recuperato non è una stringa |

Formattazione numeri, valute e date

Tutte le classi per la formattazione sono figlie della classe astratta Format e consentono di formattare una stringa in base ad uno specifico local:

- La NumberFormat per la formattazione di numeri e valute
- La DateFormat per la formattazione di data e ore
- La SimpleDateFormat è una sottoclasse di DateFormat che consente di formattare una data e un'ora in un pattern personalizzato.

N.B. Nelle formattazioni è necessario specificare il paese; perché la valuta per esempio è legata al paese e non alla lingua.

Formattazione di un numero

E' il processo che ci consente di convertire un numero in una stringa.
L'operazione inversa, invece è detta parsing di una stringa.
Entrambe le due operazioni sono dipendenti dal local

Esempio :

La stringa "1.000" in italia è 1000 dove il punto è il separatore delle migliaia

La stringa "1.000" nel Regno Unito è 1 dove il punto è il separatore dei decimali

Classe Currency

E' una classe che si occupa di formattare una stringa in una valuta in base al local. Al suo interno ha i metodi che ci recuperano informazioni sulla valuta di un local come ad esempio il simbolo €, \$ o il nome della valuta utilizzato euro, dollaro; il numero delle cifre decimali utilizzate per rappresentare un valore, ecc...

```
Currency currencyInstance = Currency.getInstance(locale);
```

Locale può essere sostituito dal codice di valuta ("EUR", "USD") dello standard ISO 4217



Classe DateFormat

Questa classe si preoccupa di visualizzare una data nel format di un paese specifico (Locale).

Solo data

```
DateFormat dateFormat = DateFormat.getDateInstance(DateFormat.FULL,  
                                                    locale);
```

Solo orario

```
DateFormat dateFormat = DateFormat.getTimeInstance(DateFormat.FULL,  
                                                    locale);
```

Sia data che ora

```
DateFormat dateFormat = DateFormat.getDateTimeInstance(DataFormat.FULL,  
                                                         DataFormat.FULL, locale);
```

Ora

```
String dateFormat.Format(data);
```

DateFormat.(**SHORT, MEDIUM, LONG, FULL E DEFAULT**)

Il **DEFAULT** corrisponde sempre al **MEDIUM**

Classe SimpleDateFormat che estende la DateFormat

Utilizzando questi simboli nel nostro pattern:

| | |
|--|--|
| G → era (BC; AD; dopo Cristo) | D → day in year (D=159; DDDD=0159) |
| y → year (yy=14; yyyy=2014; yyyyy=02014) | d → day in month (d=8; ddd=008) |
| M → month (M=6; MM=06; MMM=Jun; MMMM=June) | F → day of week in month (F=2; FF=02) |
| w → week in year (w=6; ww=06) | E → day in week (E=Sun; EEEE=Sunday) |
| W → week in month (W=1; WW=01) | u → Day number of week (value range 1-7) |

è possibile costruirsi un proprio formato di data.

Utilizzando questi simboli nel nostro pattern:

| | |
|-----------------------------|--|
| a → am/pm marker (AM; PM) | m → minute in hour (30) |
| H → hour in day 0 - 23 (11) | s → second in minute (55) |
| k → hour in day 1 - 24 (11) | S → millisecond (978) |
| K → hour in am/pm 0 - 11 | z → time zone as general time zone (Pacific Standard Time; PST; GMT-08:00) <ul style="list-style-type: none">- z returns the code (CET)- zzzz returns the description (Central European Time) |
| h → hour in am/pm 1 -12 | Z → time zone as RFC 822 time zone (-0800) |

è possibile costruirsi un proprio formato di orario.

La classe Thread

Un thread è una parte di codice che viene eseguita in concorrenza ad altre parti di codice. In realtà un programma ha già un thread singolo che si preoccupa di mandare in esecuzione il codice contenuto nel main. Quindi se creo un thread (Thread-0) e lo lancio (start()), in realtà avrò due thread in esecuzione.

Creazione di un Thread

Esistono due modi per creare un Thread:

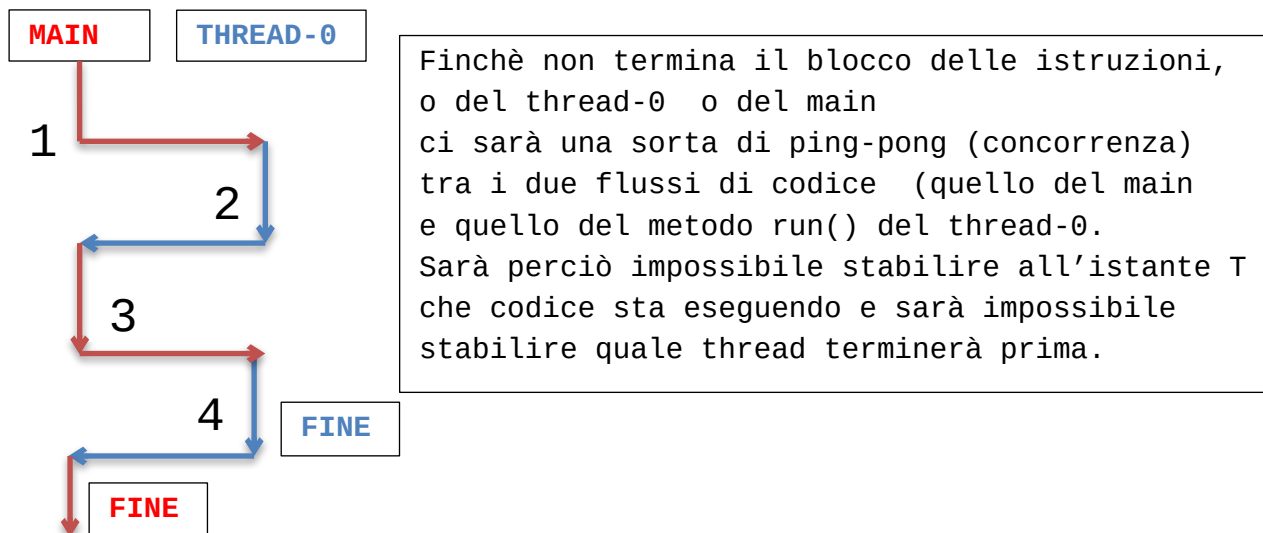
- 1) Estendendo la classe Thread
- 2) Creando una classe che implementa Runnable

Estendere la classe Thread

Nella nostra classe che estenderà la classe Thread possiamo sovrascrivere il metodo run() della classe Thread (public void run()).

Il Thread creato, thread-0, sarà in concorrenza con il nostro thread-main; quindi dopo il suo lancio (start()) sarà pronto ad essere eseguito. Sarà la JVM a decidere in completa autonomia (la priority non conta) se **proseguire con il codice restante del main o far partire il codice del metodo run() del thread-0.**

Non solo, la JVM potrà decidere di non completare il codice del Thread-0 e di ritornare al main per finire di eseguire il codice restante.



Implementare l'interfaccia Runnable

In realtà la classe Thread che estendiamo nel primo metodo implementa l'interfaccia Runnable ed la obbliga ad avere un metodo run() vuoto. Per costruire un Thread da un oggetto Runnable passiamo come parametro al costruttore del Thread la nostra istanza di Runnable.

I metodi della classe Thread, come ad esempio il metodo start() saranno utilizzati dal thread che ha il runnable che abbiamo implementato noi.

```
Thread t= new Thread(Runnable r);
```

```
r.start(); // non esiste il metodo start nella classe Runnable
```

```
t.start();
```

N.B. Si preferisce creare Thread implementando l'interfaccia Runnable perché Java permette l'implementazione di più interfacce, invece non consente l'ereditarietà multipla

N.B. Sono da considerare Thread di un programma tutti quelli pronti ad essere eseguiti. Un Thread diventa pronto all'esecuzione solo dopo che lo abbiamo lanciato con il metodo start().

Thread e l'InterruptedException

Un metodo della classe figlia non può lanciare un'eccezione più ampia di un metodo della classe padre; perciò i metodi sottoclasse della classe Thread lanceranno solo eccezioni di tipo InterruptedException o sue sottoclassi.

I metodi della classe Thread

I metodi della classe thread ci consentono:

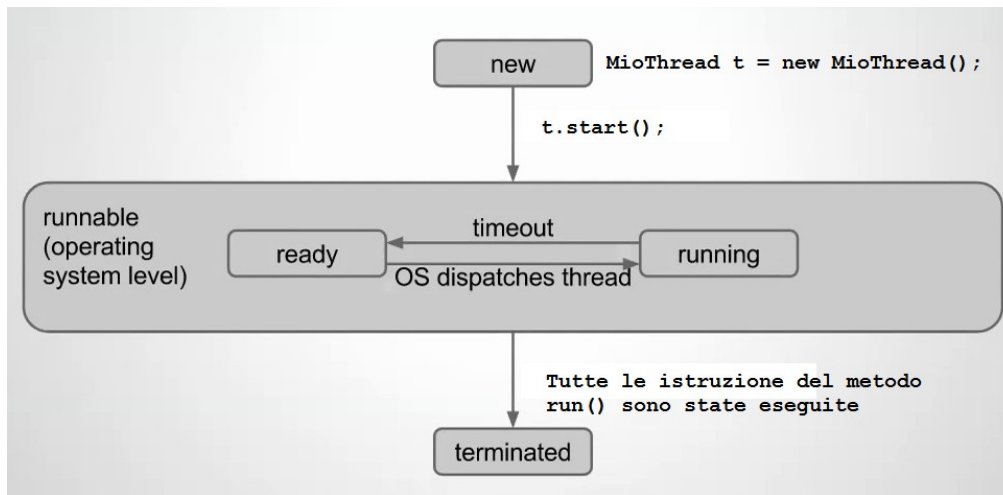
- Di avere un margine di controllo del thread
- di sapere lo stato del thread in quel dato momento

Esecuzione asincrona dei Thread

Se nel nostro programmino abbiamo n thread e non esiste nessun meccanismo di sincronizzazione tra loro, per ogni esecuzione del nostro programma possiamo avere n possibili risultati imprevedibili perché tutto dipende dal ping pong che la JVM farà da un thread ad un altro (il main è anch'esso un thread), esecuzione asincrona non deterministica.

Ciclo di vita del Thread

Un thread ha un ciclo di vita, nasce quando lanciamo il thread (start()) e muore con l'ultima istruzione del metodo run(). Queste sono le uniche fasi sicure del thread. Possiamo sapere lo stato di un Thread, tuttavia non è il caso di basarsi sullo stato(getState()) perché già nel momento in cui andiamo a recuperarlo in realtà potrebbe essere cambiato.



Esecuzione
Asincrona

Data races

Letteralmente è una corsa ai dati, si verifica quando due o più thread provano ad accedere ad una variabile comune e non ci consente di avere un controllo sulla modifica incondizionata della stessa dai vari thread.

Soluzione data races

Quello che dobbiamo fare è assicurarci che solo un thread modifichi una variabile in un'unica operazione atomica senza essere disturbato da altri thread. Questa sezione di codice è detta sezione critica. Il lock è il testimone che un thread al momento sta eseguendo la sezione critica mentre gli altri sono in attesa che il lock venga rilasciato e liberi la sezione critica dal corrente thread.

Esecuzione sincrona dei thread

La sezione critica è un blocco di istruzioni o un intero metodo che deve essere eseguito da un singolo thread (il thread che ha il lock).

Sincronizzazione di un blocco

Il lock può essere acquisito su un oggetto, un thread deve prima acquisire il lock e dopo può eseguire il codice al suo interno.

Una volta terminata la sezione critica il lock viene rilasciato e un altro thread può acquisire il lock.

ESECUZIONE SINCRONA

```
public void increment() {
    synchronized (this) {
        count++;
        System.out.print(count + " ");
    }
}
```

Sezion
e
critica

UN SOLO THREAD ESEGUIRÀ QUESTE DUE ISTRUZIONI COME UN'UNICA OPERAZIONE

ESECUZIONE ASINCRONA

```
public void increment() {
    count++;
    System.out.print(count + " ");
}
```

Sezion
e
critica

MENTRE INCREMENTO IL VALORE DI COUNT UN ALTRO THREAD ESEGUIRÀ LO STESSO CODICE E CAMBIERÀ IL VALORE DI COUNT PRIMA CHE SIA STAMPATO.

N.B. this è solo un testimone, può essere qualsiasi oggetto, o anche un oggetto.class; è però importante che sia lo stesso oggetto per tutti i thread. Se per esempio, il metodo increment() viene richiamato da istanze diverse, this è diverso per ogni istanza.

Sincronizzazione di un metodo

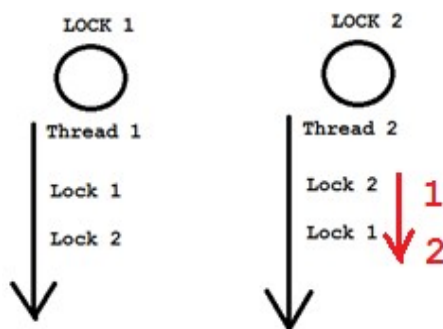
Un metodo dinamico sincronizzato è come un blocco synchronized su tutto il codice contenuto nel metodo ed ha come lock l'oggetto istanziato che è rappresentato dal this; così come un metodo statico è come un blocco synchronized su tutto il codice contenuto nel metodo statico che ha come lock l'oggetto.class associato alla classe del metodo.

N.B. non ha senso sincronizzare tutto il codice contenuto nel metodo se in realtà è sufficiente che solo poche istruzioni siano eseguite da un solo thread alla volta (no ping pong).

Tutti i thread che sono in attesa che un lock venga rilasciato saranno in un nuovo stato quello di suspended.

Problemi dei locks dei thread

DeadLocks



- 1) facciamo partire i Thread 1 e 2
- 2) Il thread 1 acquisisce il lock 1
- 3) Il thread 2 acquisisce il lock 2
- 4) Il thread 1 non può acquisire il lock 2 perché è bloccato dal Thread 2
- 5) Ma Il thread 2 non rilascerà mai il lock 2 perché per farlo deve aspettare che lock 1 venga rilasciato

Conclusione

- 6) Thread 1 aspetta che il lock 2 sia rilasciato
- 7) Thread 2 aspetta che il lock 1 sia rilasciato

Lo scenario è che il lock non verrà mai rilasciato e sia il thread1 che il thread2 resterà per sempre in attesa. (quadrato rosso in eclipse). Il lock si presenta quando abbiamo 2 o più lock.

La soluzione è far acquisire i lock da tutti i thread nello stesso ordine.

N.B. E' possibile che il deadlock non si presenta, tutto dipende da come i thread acquisiranno i lock durante l'esecuzione.

Live lock

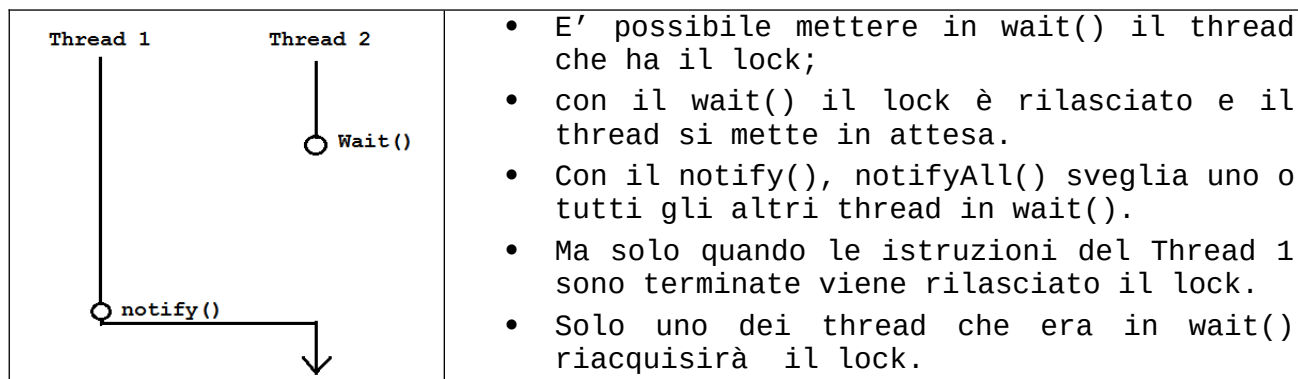
I Thread sono sempre in esecuzione ma non riescono ad andare avanti.

Lock starvation

il thread a bassa priorità non viene mai eseguito perché ci sono n thread tutti a priorità alta

Meccanismo Wait/Notify

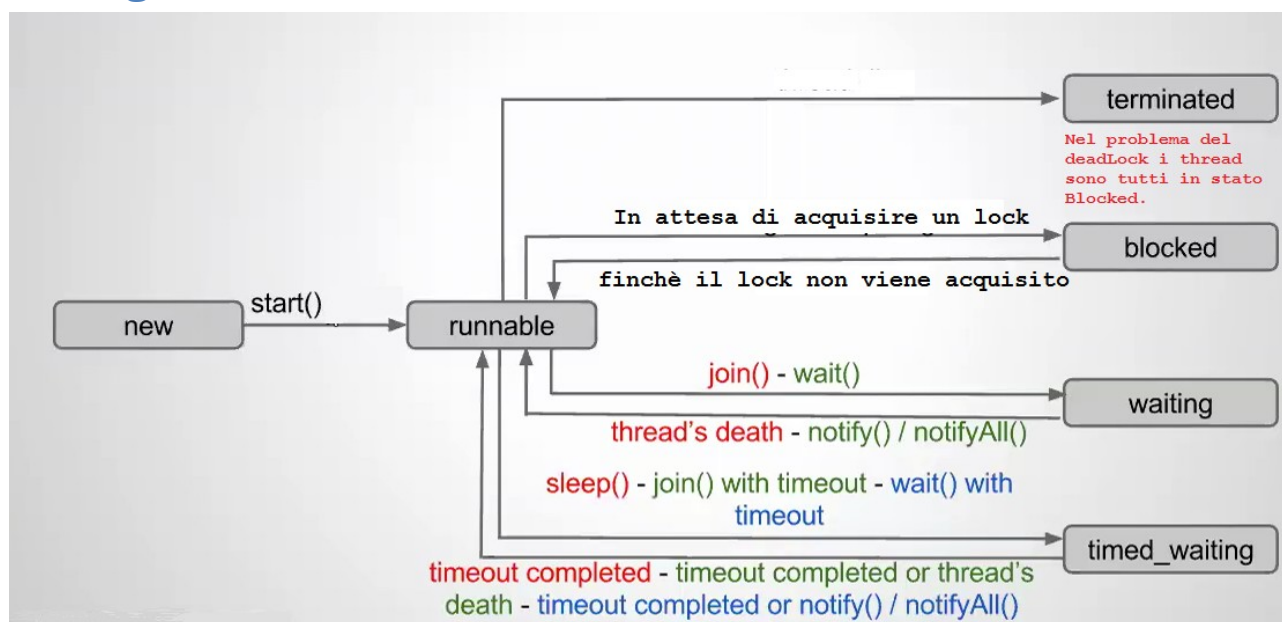
Il meccanismo di wait/notify consente a due thread di comunicare tra loro. Se un thread chiama il metodo wait() si mette in attesa finchè un altro thread chiama il notify()/notifyAll().



N.B. E' possibile chiamare wait/notify/notifyAll solo sul Thread che ha acquisito il lock, altrimenti si ha un illegalMonitorStateException.

N.B. Il notify()/notifyAll non rilasciano il lock.

Tutti gli stati del Thread



IllegalThreadStateException

Questa eccezione viene lanciata quando un thread non è in un stato appropriato per l'operazione richiesta (esempio start() su un thread già lanciato start()). E' una RuntimeException, ed è un errore di programmazione da evitare.

Package java.util.concurrent

La gestione dei Thread così come l'abbiamo studiata nel capitolo precedente diventa alquanto complessa quando aumentano i numeri dei thread.

La classe concurrency introdotta nel java 5 ci consente di avere una gestione semplificata dei thread.

La classe Synchronizer

Questa classe astratta consente di sincronizzare due o più thread.

Ci sono più sottoclassi concrete della classe synchronizer, ognuna identifica un differente modo di sincronizzare i thread:

- Semaphore
- Phaser
- CountdownLatch
- Exchanger
- CyclicBarrier

Semaphore

Nella gestione dei thread del capitolo precedente, una sezione critica poteva essere eseguita ad un solo thread.

Il semaphore mantiene un contatore del numero di risorse disponibili, in tal modo se ha 3 risorse allora 3 thread possono acquisire la risorsa.

Acquisire la risorsa significa ottenere il lock sull'oggetto e quindi il thread che ha acquisito la risorsa potrà accedere alla sezione critica.

Se il semaphore ha un contatore uguale zero allora non ci sono risorse, il thread resterà in attesa (blocked) che si liberi una risorsa.

Se il semaphore ha un contatore maggiore di zero, il valore restituito indica il numero di risorse disponibili e quindi il numero di thread che possono acquisire la risorsa.

Un Thread acquisisce una risorsa con il metodo acquire() (che sarebbe come acquisire il lock su una sezione critica) e la rilascia con il metodo release().

```
Public void run() {  
    Semaphore s=new Semaphore(2);  
    mioThread.semaphore.acquire();  
    .....//sezione critica  
    mioThread.semaphore.release();  
}
```

N.B. Se il thread non rilascia la risorsa, il thread avrà il lock per sempre (deadLock).

N.B. La nostra sezione critica è tra il metodo acquire() ed il metodo release().

CountDownLatch

Tutti i thread sono in attesa che un contatore completi il conteggio. Quando il countdown arriva a zero tutti i thread che fanno uso del countDownLatch e che hanno chiamato il metodo await() partiranno.

```
Public void run() {  
    CountDownLatch cdl= new CountDownLatch(5);  
    mioThread.cdl.await();  
}
```

Exchanger

I thread si scambiano un messaggio;



- 1) Il thread 2 chiama il metodo exchange(), invia al thread 1 un oggetto e si mette in attesa.
- 2) Finchè il thread 1 non chiama anche lui il metodo exchange() per scambiare il messaggio.
- 3) Il messaggio è una qualsiasi reference indicata come generics

```
Exchange<String> talk = new Exchange<String>();
```

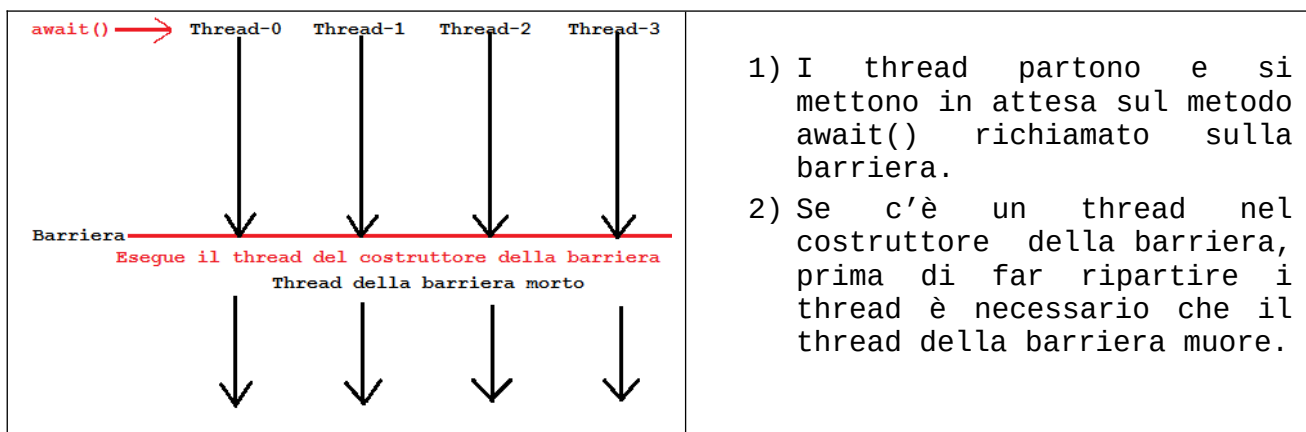
N.B. Se il thread 1 non chiama il metodo exchanger(); il thread 1 termina il suo codice mentre il thread 2 resta in attesa per sempre (deadLock), sempre se non specifico un timeout.

CyclicBarrier

Nel countDownLatch tutti thread attendono il decremento del countdown chiamando il metodo await() sulla nostra classe countDownLatch, quando il countdown arriva a zero un thread che parte in ritardo non attenderà alcun countdown.

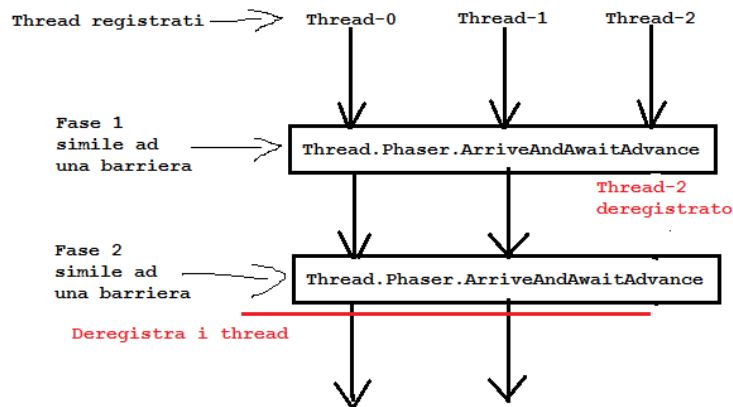
Nel CyclicBarrier, invece si aspetta realmente che n thread partano e sono in attesa al metodo await().

Se non si raggiunge gli n thread, i thread che sono in attesa, resteranno in attesa (situazione di deadlock).



Phaser

Viene utilizzato quando vogliamo che tutti i thread insieme svolgano dei compiti intermedi, una volta che tutti hanno completato questo compito, potranno ripartire e proseguiranno per raggiungere un risultato finale.



- I Thread che partecipano al phaser devono registrarsi, in modo da sapere il numero di thread da aspettare.
- Tutti i thread registrati devono eseguire la fase 1 (codici di istruzioni) ed avvisare che è stata completata con il metodo ArriveAndAwaitAdvance
- Quando il metodo ArriveAndAwaitAdvance è stata chiamata da tutti, i thread attualmente registrati.
- Una volta terminate tutte le fasi i thread possono deregistrarsi

Thread-safe

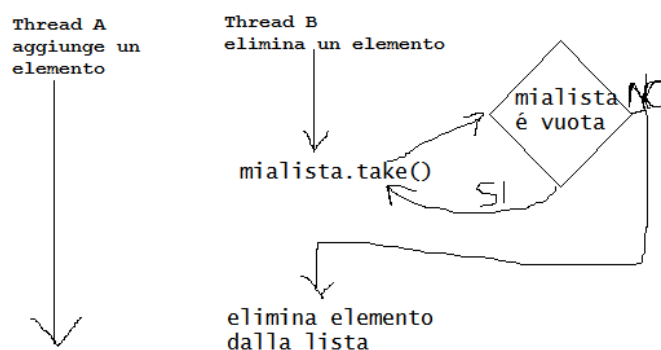
Per ogni collection di java esiste una versione Thread safe che consente di gestire i thread in una collection evitando il problema del data race.

Esempio:

Java.util.HashMap Java.util.concurrent.ConcurrentHashMap

Se abbiamo due thread, uno che aggiunge un elemento in una coda, ed uno che lo elimina. Il thread che vuole eliminare un elemento da una lista vuota resterà in attesa che si aggiunga un nuovo elemento nella lista, non appena è disponibile il metodo take() si sblocca ed eliminerà l'elemento dalla lista.

PriorityBlockingQueue<Integer> mialista



Variabili atomiche

Anche qui viene evitato il problema del data race anche se non è utilizzato il blocco synchronized. Per ogni Wrapper classe esiste una corrispettiva classe atomica.

| Class (extend from Object unless noted otherwise) | Short description |
|---|--|
| AtomicBoolean | Atomically updatable Boolean value |
| AtomicInteger | Atomically updatable int value; inherits from the Number class |
| AtomicIntegerArray | An int array in which elements can be updated atomically |
| AtomicLong | Atomically updatable long value; inherits from Number class |
| AtomicLongArray | A long array in which elements can be updated atomically |
| AtomicReference<V> | An atomically updatable object reference of type V |
| AtomicReferenceArray<E> | An atomically updatable array that can hold object references of type E (E refers to be base type of elements) |

Questa classe mette a disposizione una serie di metodi che ci consentono di fare operazioni sulla classe stessa da un singolo thread.

Se per esempio dobbiamo stampare un intero incrementato e dobbiamo assicurarci che l'incremento venga fatto prima della stampa allora dobbiamo essere certi che nessun altro thread stampi prima dell'incremento.

